Tourist Management System

A Report

Submitted in partial fulfillment of the requirement for the degree of **B.Tech.**

in

Computer Science & Engineering

Under the Supervision of

Ms. Renu (Assistant Professor)

by

Avdhesh kumar (2304231530010)
Akhil Mishra (2204231530004)
Hariom Chaurasiya (22042315300013
Durgesh Singh (2204231630008)

Department of Computer Science & Engineering
School of Management Sciences, Lucknow

Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow

2025 - 2026

CERTIFICATE

This is to certify that Project Report entitled "**Tourist Management System**" which is submitted by Avdhesh Kumar, Akhil Mishra, Hariom Chaurasiya and Durgesh Singh in partial fulfilment of the requirement for the award of degree B.Tech. in Department of Computer Science and Engineering of School of Management Sciences Lucknow, affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidates own work carried out by them under my/our supervision. The project embodies result of original work and studies carried out by the students themselves and the contents of the project do not form the basis for the award of any other degree to the candidate or to anybody else.

Signature:

Mr. Sunit Kumar Mishra

Head of Department
CSE Department,
SMS, Lucknow

Signature:

Ms. Renu

Assistant Professor
CSE Department,
SMS, Lucknow

Date:

DECLARATION

We hereby declare that this submission is our own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

All the sources of information and references used in this project have been duly acknowledged. Any assistance received during the course of this work has been properly credited. I take full responsibility for the authenticity of the content and results presented in this project.

Name: Avdhesh kumar
Roll No.: 2204231530010

Name: Akhil Mishra
Roll No: 2204231530004

Name: Hariom Chaurasiya
Roll No: 22042315300013

Name: Durgesh Singh
Roll no: 2204231630008

Date:

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the B.Tech. Project undertaken during B.Tech. Final Year. We owe special debt of gratitude to our project supervisor Ms. Renu, Department of Computer Science and Engineering, School of Management Sciences Lucknow, for his constant support and guidance throughout the course of our work. His sincerely, thoroughness and perseverance have been a constant source of inspiration for us. It is only his cognizant efforts that our endeavors have seen light of the day.

We take this opportunity to acknowledge the valuable contribution of Professor Mr. Sunit Kumar Mishra Sir, Head, Department of Computer Science & Engineering, School of Management Sciences, Lucknow, for his full support, guidance, and continuous encouragement during the development of the project. We also express our sincere gratitude to **Dr. Niyati Gaur**, Assistant Professor, Department of Computer Science & Engineering and Project Coordinator, for her constant guidance, motivation, and insightful suggestions, which greatly contributed to the successful completion of this project.

We also do not like to miss the opportunity to acknowledge the contribution of all faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

Name: Avdhesh kumar
Roll No: 2204231530010

Name: Akhil Mishra
Roll No: 2204231530004

Name: Hariom Chaurasiya
Roll No: 2204231530013

Name: Durgesh Singh
Roll no: 2204231530008

Date:

ABSTRACT

The **Tourist Management System** is a web-based application developed using **Python and the Django framework** to simplify and improve the management of tourism-related services. In the modern travel industry, tourists often face difficulties in obtaining reliable information about destinations, travel packages, accommodations, and booking services. The absence of a centralized platform can lead to inefficient planning, misinformation, and inconvenience for travelers.

The proposed system provides a centralized digital platform where tourists can easily explore tourist destinations, view available travel packages, and make bookings online. Users can register on the platform, browse destinations, check details such as location, price, available facilities, and accommodation options, and select suitable packages according to their preferences and budget.

The system also includes an **administrative module** that allows administrators to manage tourist information, travel packages, hotel details, and booking records efficiently. Through the admin dashboard, administrators can update destination details, monitor user bookings, and maintain accurate and up-to-date information on the platform.

The application implements **secure authentication, user profile management, and role-based access control** to ensure safe and reliable system usage. The system is designed to be **user-friendly, scalable, and accessible across different devices**, making it convenient for users to plan their trips anytime and anywhere.

Overall, the Tourist Management System aims to enhance the tourism experience by providing a simple, organized, and efficient solution for managing tourist information and travel services, while also helping administrators manage tourism operations more effectively.

LIST OF FIGURES

LIST OF ABBREVIATIONS

➢ RAM	---	Random Access Memory
➢ OS	---	Operating System
➢ SSD	---	Solid State Drive
➢ UI/UX	---	User Interface/ User Experience
➢ API	---	Application Programming Interface
➢ CSRF	---	Cross-Site Request Forgery
➢ IDE	---	Integrated Development Environment
➢ HTTP	---	Hyper Text Transfer Protocol
➢ SMTP	---	Simple Mail Transfer Protocol
➢ CRUD	---	Create Read Update Delete

TABLE OF CONTENTS

Page

CHAPTER 1

INTRODUCTION

1.1. Literature Review

In today's fast-paced world, the tourism industry has experienced significant growth due to increased accessibility of travel services and digital technologies. Tourists often rely on various online platforms to gather information about destinations, accommodation, travel packages, and transportation services. However, many travelers still face difficulties in finding reliable, well-organized, and centralized information when planning their trips. Traditional methods such as travel agencies, brochures, and scattered online sources often lack updated information, structured communication, and convenient booking facilities. These limitations can lead to confusion, inefficient trip planning, and inconvenience for travelers.

To overcome these challenges, digital tourism management platforms have been developed to simplify the process of travel planning and booking. Modern tourism applications allow users to explore destinations, check available travel packages, and book services through a single integrated platform. These systems improve accessibility and convenience by providing tourists with accurate information, interactive features, and efficient booking mechanisms.

Several existing travel platforms such as **MakeMyTrip, Booking.com, TripAdvisor, and Airbnb** provide online services for travel planning and accommodation booking. These platforms offer features such as hotel reservations, travel package booking, destination reviews, and price comparisons. While these systems provide useful travel services, many of them focus mainly on booking and reviews rather than providing a comprehensive tourism management system that integrates destination information, package management, and administrative control within a single platform.

Considering these limitations, the proposed **Tourist Management System** aims to provide a centralized and efficient web-based solution for managing tourism-related services. The system allows users to browse tourist destinations, view detailed information about travel packages, and make bookings through a user-friendly interface. Administrators can manage destinations, tour packages, and booking records through a centralized dashboard.

By utilizing modern web technologies such as **Python, Django, PostgreSQL, and Bootstrap**, the system ensures better scalability, security, and ease of use. The proposed solution aims to enhance the overall travel planning experience while enabling efficient management of tourism services through a reliable digital platform.

1.2. Problem Definition

Planning a trip and managing tourism-related activities can often be a complex and time-consuming process. Tourists frequently face difficulties in finding reliable information about destinations, travel packages, accommodations, and transportation services. The absence of a centralized and organized platform makes it challenging for travelers to plan their journeys efficiently.

In the existing scenario, tourists usually depend on multiple sources such as travel agencies, brochures, different websites, or social media platforms to gather travel information. These methods may provide useful information, but they often suffer from several limitations:

- **Lack of centralized information:** Travel details such as destinations, packages, accommodation, and pricing are scattered across different platforms, making it difficult for users to access complete information in one place.
- **Limited filtering and search options:** Many traditional platforms do not allow users to easily search and filter travel options based on criteria such as location, budget, travel duration, or type of destination.
- **Manual booking process:** In many cases, tourists still need to contact travel agencies or service providers manually, which increases the time and effort required for trip planning.
- **Data inconsistency and outdated information:** Information available on different sources may not always be updated, which can lead to confusion or incorrect travel decisions.
- **Lack of secure user management:** Many systems do not provide proper authentication or user account management, which may affect data privacy and security.

To overcome these challenges, the **Tourist Management System** is proposed as a web-based solution that provides a centralized platform for managing tourist destinations, travel packages, and booking services. The system aims to simplify the travel planning process by offering organized information, efficient booking mechanisms, and secure user management features.

1.3. Brief Introduction of Project

The **Tourist Management System** is a web-based application developed to simplify and digitalize the process of exploring tourist destinations, managing travel packages, and booking tourism services. The system provides a centralized platform where users can create an account, manage their personal profiles, browse tourist destinations, and book travel packages in a structured and convenient manner. By providing all necessary travel information in one place, the system reduces the dependency on traditional travel agents and scattered online resources.

The application is developed using Python and the Django framework, along with PostgreSQL for database management and Bootstrap for responsive user interface design. These technologies provide scalability, secure data handling, and an interactive web interface for users. Through the platform, tourists can easily view detailed information about destinations such as location, description, travel cost and available facilities.

To improve usability and user experience, the system provides search and filtering features that allow tourists to find destinations based on criteria such as location, budget, and type of travel package. Users can also maintain their personal profiles and view their booking history through the system.

Overall, the Tourist Management System aims to provide a user-friendly, efficient, and reliable platform for managing tourism services. The system simplifies travel planning for tourists while enabling administrators to efficiently manage tourism data through a secure and scalable web-based application.

1.4. Proposed Modules

The Tourist Management System integrates modern web technologies and database management techniques to simplify the process of exploring tourist destinations and managing travel bookings. The system is designed to provide a reliable, user-friendly, and scalable platform where users can search tourist destinations, view travel packages, and book trips easily. The architecture of the system follows a modular design consisting of the following primary modules:

1. User Interface Module:

The frontend of the system is developed using **HTML, CSS, and Bootstrap**, which provides a responsive, clean, and interactive user experience. The interface allows users to easily navigate through the platform and access tourism-related information. Through the user interface, users can:

- Create and manage user accounts
- Browse and explore different tourist destinations
- Search destinations using filters such as location, budget, and travel type
- View detailed destination information including description, price, and facilities

The responsive design ensures that the system works smoothly on desktops, tablets, and mobile devices, improving accessibility and usability for all users.

2. Tourist Destination Management Module

This module allows the administrator to manage information related to tourist destinations. The admin can add, update, or delete destination details such as destination name, location, description, images, and travel cost. This ensures that users always have access to updated and accurate travel information.

3. Tour Package and Booking Module

This module allows tourists to view available tour packages and make bookings through the platform. Users can select travel packages according to their preferences and submit booking requests. The system records and manages all booking details for future reference.

4. User Authentication and Profile Module

The system provides secure **user registration and login functionality**. Users can create accounts, log in securely, and manage their personal profiles. The module also allows users to view their booking history and update personal details when necessary.

5. Admin Management Module

This module enables administrators to monitor and manage the overall system. Through the admin dashboard, administrators can manage users, tourist destinations, travel packages, and booking records. This helps maintain system accuracy and smooth operation.

2. Backend and Inference Pipeline:

The backend of the Tourist Management System is implemented using Python with the Django framework, which handles the business logic, server-side processing, and communication with the database. Django follows the Model–View–Template (MVT) architecture, making the system well-structured, secure, and scalable:

This layer is responsible for:

- **User authentication and session management**, allowing users to securely register and log in to the system.
- **Tourist destination management**, including adding, updating, and deleting destination information.
- **Tour package and booking management**, enabling users to select travel packages and submit booking requests.

The system processes user requests and handles server-side logic to display tourism data dynamically.

Django's built-in security features help protect the system from common vulnerabilities and ensure smooth interaction between the frontend interface and the database.

3. Database and Authentication Layer:

The database layer uses **PostgreSQL**, a powerful and reliable relational database management system, to store and manage system data securely.

- User account details and profile information
- Tourist destinations and travel package data
- Booking records and transaction details
- Administrative records and system logs

Django's **Object Relational Mapping (ORM)** framework is used to interact with the PostgreSQL database, allowing efficient data retrieval, insertion, and updates without writing complex SQL queries.

To maintain system security and data integrity, the application implements secure authentication, role-based access control, and protected database access mechanisms. These measures ensure that sensitive user data remains secure while maintaining reliable system

performance.

1.5. Hardware Requirements

1.5.1. Minimum Requirements

A computer with an **Intel i3 processor (10th generation or equivalent)**, **4 GB RAM**, and at least **100 GB of storage** is required to run the Tourist Management System efficiently during development and testing. The use of a **Solid-State Drive (SSD)** is preferred over a traditional hard disk drive because it significantly improves system boot time and application performance The application can function properly on a display with a minimum resolution of **1366 × 768 pixels**. A basic **internet connection** is required for installing development tools, downloading dependencies, and accessing online resources. Standard input devices such as a **keyboard and mouse** are necessary for system interaction. These minimum specifications are sufficient for basic development, testing, and demonstration of the system.

1.5.2. Recommended Requirements

For optimal performance, it is recommended to use a system with an Intel i5 or i7 processor (11th generation or higher), at least 8 GB of RAM, and a 250 GB SSD. Higher RAM capacity improves multitasking efficiency while running development servers, databases, and code editors simultaneously. A Full HD (1920 × 1080) display enhances UI/UX design, debugging, and overall visual clarity. A stable high-speed internet connection is essential for continuous interaction with cloud-based services such as SQLite, API integrations, and deployment platforms. Additional peripherals such as a webcam and microphone may support future enhancements like video verification or real-time communication features. Overall, these specifications ensure a smooth development experience and reliable system performance.

1.6. Software Requirements

1.6.1. Frontend Technologies

The frontend of the Tourist Management System is developed using modern web technologies such as HTML, CSS, and Bootstrap, which enable the creation of responsive and user-friendly web interfaces. Bootstrap is used to ensure consistent styling and faster UI development across different screen sizes and devices. These technologies allow users to easily navigate through the system, explore tourist destinations, view travel packages, and manage bookings through a clean and interactive interface.

1.6.2. Backend and Database

The backend of the system is developed using **Python with the Django framework**, which provides a secure and scalable environment for handling application logic and user requests. Django follows the **Model–View–Template (MVT)** architecture, making the system well structured and maintainable.
For database management, **SQLite** is used as the primary database during development. SQLite stores important data.

1.6.3. Authentication and Mailing Services

Django provides a built-in authentication system that manages user registration, login, and session handling securely. The system ensures that only authorized users can access specific functionalities through role-based access control. Email services such as account verification, password reset requests, and notification messages can be implemented using Django's email framework. These services improve system usability and enhance communication between the system and users.

1.6.4. Development Tools

The development of the system is carried out using **Visual Studio Code (VS Code)**, which is a powerful and widely used integrated development environment (IDE). Version control and collaboration are managed using **Git and GitHub**, which help developers track changes, maintain source code history, and work efficiently in teams.

1.6.5. Browser and Operating System Compatibility

The Tourist Management System is designed to run smoothly on modern web browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge. The system can be developed and tested on major operating systems including Windows 10/11, Linux (Ubuntu), and macOS, ensuring cross-platform compatibility for both developers and users.

CHAPTER 2
SYSTEM ANALYSIS AND SPECIFICATION

2.1. Functional Model

The functional model of the Tourist Management System describes the operations performed by the system based on its functional requirements. It explains how different system processes interact with external entities such as tourists (users) and administrators, how data flows through the system, and how information is stored and retrieved from the database.

2.1.1. 0 Level DFD

The **0-Level Data Flow Diagram (Context Diagram)** represents the overall interaction between external entities and the Tourist Management System. In this diagram, the entire system is represented as a single process called *Tourist Management System*. The two main external entities interacting with the system are **Tourist (User)** and **Administrator**.

The tourist sends requests such as searching for destinations, viewing tour packages, and booking tours. In response, the system provides information about destinations, travel packages, and booking confirmations.

The **administrator** manages system data such as tourist destinations, travel packages, and booking records. The system also generates reports and system information for the administrator.

Fig 2.1. 0 Level DFD

2.1.2. 1 Level DFD

The 1-Level Data Flow Diagram (DFD) provides a detailed representation of the internal processes of the Tourist Management System. In this diagram, the main system process is divided into several sub-processes such as User Authentication, Destination Management, Tour Package Management, and Booking Management.

The Tourist (User) interacts with the system by registering, logging in, searching for tourist destinations, viewing tour packages, and making bookings. The Administrator manages the system by adding or updating destination information, managing tour packages, and monitoring booking records.

All system processes interact with the **Tourism Database**, which stores information related to users, destinations, tour packages, and booking details. This diagram helps in understanding how data flows between users, administrators, system processes, and the database within the Tourist Management System.

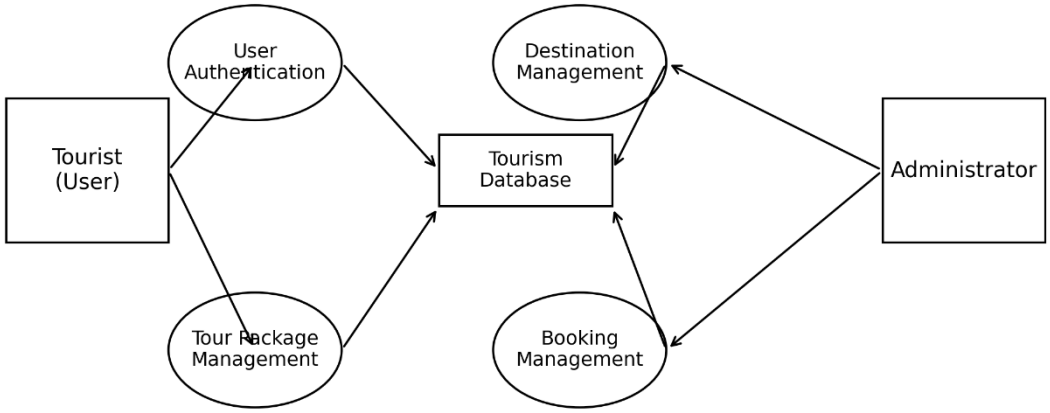

Fig 2.2. 1 Level DFD

2.2. Data Model

The User entity stores information about tourists and administrators. The Destination entity contains details of tourist places. Users can make reservations through the Booking entity, and each booking is associated with a Payment record. Users can also provide feedback through the Review entity and submit issues through the Complaint entity. These entities and relationships help organize and manage tourism data effectively.

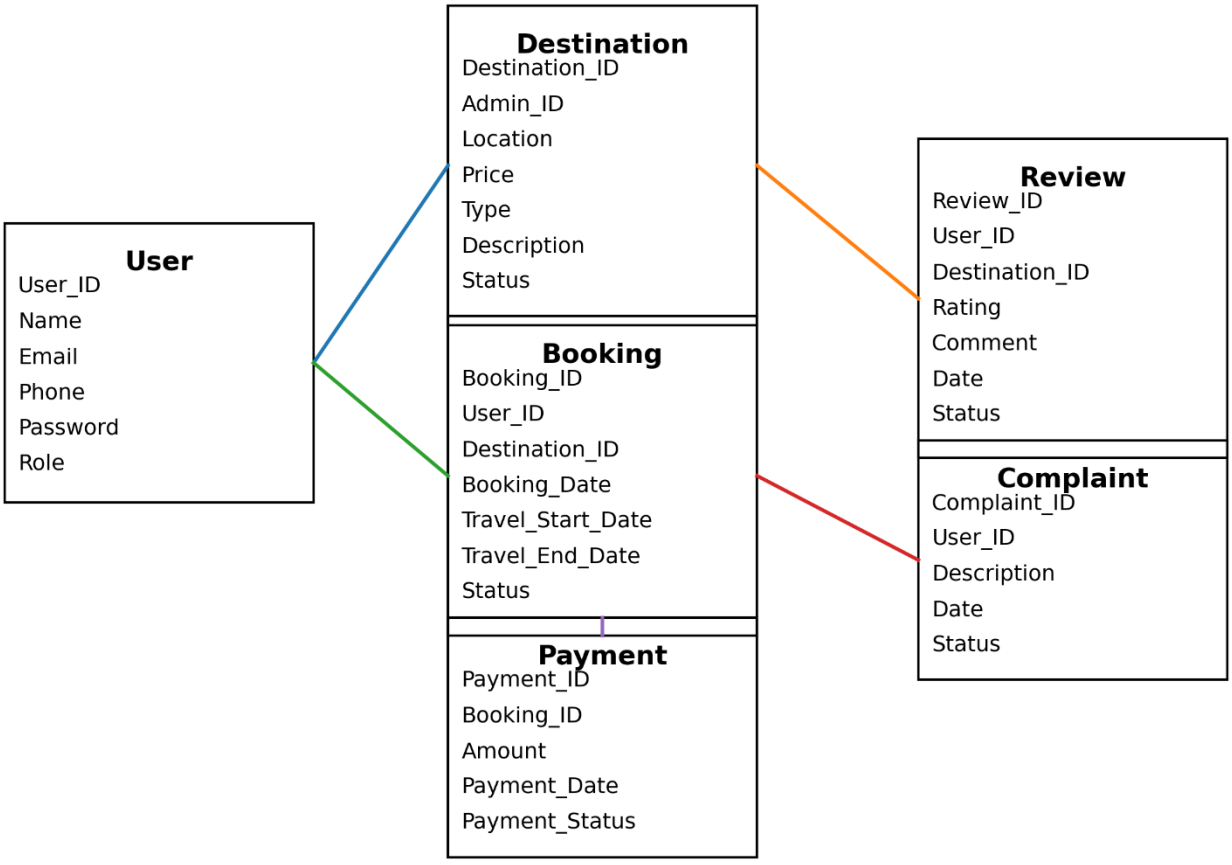

Fig 2.3. E-R Diagram

2.3. Process Flow Model

The process flow diagram shows how the Tourist Management System operates. The user opens the system and logs in or registers. After successful authentication, tourists can search destinations, view tour packages, and send booking requests. The administrator manages destinations and approves or rejects booking requests. If the request is approved, the payment process is completed and the system sends a booking confirmation to the user.

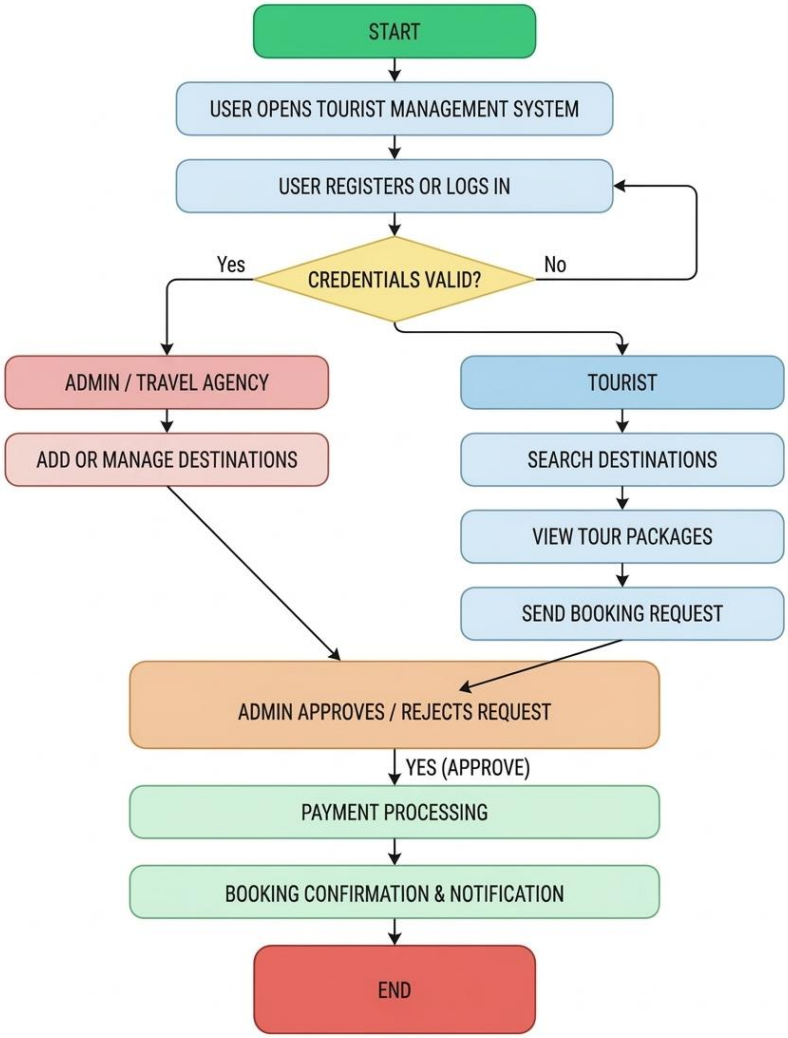

Fig 2.4. Process Flow Diagram

2.4. User Data Model

The User Data Model is an important component of the Tourist Management System that stores information related to system users such as tourists and administrators. It contains essential user details including User ID, name, email, phone number, and password, which are used for authentication and account management.

The role attribute (Tourist or Admin) helps the system provide role-based access to different functionalities. Tourists can search destinations, view tour packages, and make bookings, while administrators manage destinations, bookings, and system records.

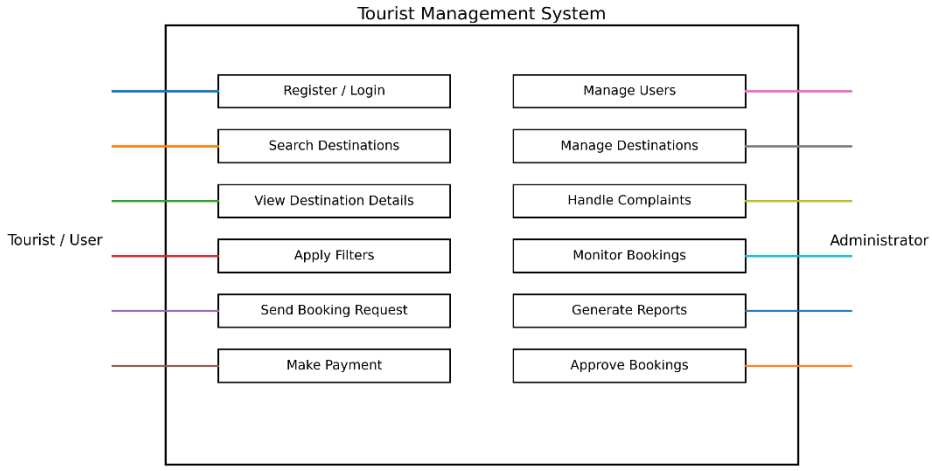

Fig 2.5. Use Case Diagram

2.5. System Design

The system architecture of the Tourist Management System is designed using a client–server model, which ensures efficient communication between users and the application. The architecture mainly consists of two major layers: the Presentation Layer (Frontend) and the Application Layer (Backend) along with the Database Layer for storing system data.

The layered design enables a separation of concerns, making the system easy to maintain, test, and upgrade. This modular approach also ensures that changes or improvements in one layer (for example, upgrading the AI model) do not disrupt the functionality of other layers

2.5.1. Technical Feasibility

Technical feasibility determines whether the proposed Tourist Management System can be successfully developed using available technologies, tools, and infrastructure.
The system is developed using modern and widely supported technologies that ensure reliability, scalability, and efficient performance.

- **Frontend:** The user interface is developed using HTML, CSS, and Bootstrap, which provide a responsive and user-friendly design. Bootstrap helps in creating consistent layouts and ensures compatibility across different devices and screen sizes.
- **Backend**: The backend is implemented using Python with the Django framework. Django provides built-in features such as authentication, URL routing, and database management, which simplify the development of secure and scalable web applications.
- **Database:** The system uses SQLite as the database for storing application data. SQLite is lightweight, reliable, and suitable for small to medium-sized web applications.
- **Development Tools**: The system is developed using tools such as Visual Studio Code, Python, and Git for efficient coding, debugging, and version control.
- **Hosting and Deployment:** The application can be deployed on web servers or cloud platforms such as PythonAnywhere or Heroku, allowing users to access the system through a web browser.

2.5.2. Operational Feasibility

Operational feasibility evaluates how effectively the Tourist Management System will function in real-world conditions and how easily users can interact with it. The system is designed with a focus on simplicity, accessibility, and user-friendly navigation for both tourists and administrators.

The application provides a responsive web interface that can be accessed through any modern web browser on desktops, tablets, or mobile devices. Users can easily register, log in, search tourist destinations, view tour packages, and make bookings without requiring advanced technical knowledge. The administrator can efficiently manage tourist destinations, booking requests, and system data through the administrative interface.

The system requires minimal setup because it operates through a web browser and stores data

in a centralized database. Secure authentication ensures that only authorized users can access system features. By automating tasks such as destination management, booking processing, and payment handling, the system reduces manual effort and operational errors.
Overall, the Tourist Management System can be easily adopted by travel agencies, tourists, and tourism service providers, making it operationally feasible for real-world usage.

2.5.3. Economic Feasibility

Economic feasibility evaluates whether the proposed Tourist Management System is cost-effective and financially practical compared to the resources required for development and maintenance.

The system is designed using open-source technologies, which significantly reduce development costs. Technologies such as Python, Django, HTML, CSS, and Bootstrap are freely available and do not require licensing fees. The database SQLite is lightweight and free to use, making it suitable for academic projects and small-scale applications.

Since the application is web-based, it can be deployed on low-cost hosting platforms or cloud services. Development tools such as Visual Studio Code and Git are also open-source, which further reduces the overall project cost.

The system requires minimal hardware resources and maintenance expenses. As a result, the Tourist Management System is economically feasible for educational purposes, small travel agencies, and startup tourism platforms.

- **Hosting on Vercel:** The Tourist Management System can be hosted on low-cost or free hosting platforms such as PythonAnywhere or Heroku, which support Python and Django applications. These platforms provide easy deployment and allow users to access the system through a web browser.

- **Database:** The system uses SQLite, a lightweight and open-source database that efficiently stores user data, tourist destination details, and booking records. SQLite requires minimal configuration and is suitable for small to medium-sized applications.

- **Development Tools:** Technologies such as Python, Django, HTML, CSS, and Bootstrap are open-source and freely available. Development environments like Visual Studio Code and version control tools such as Git help manage the project efficiently without additional licensing costs.

The combination of these technologies results in low development and operational costs, making the Tourist Management System economically feasible for academic projects, small travel agencies, and startup tourism platforms. The system can also be easily scaled in the future if the number of users or tourism services increases.

CHAPTER 3

MODULE IMPLEMENTATION AND SYSTEM INTEGRATION

This chapter explains the detailed implementation of the various modules of the Tourist Management System and how these modules work together as an integrated application. The system follows a modular architecture, where each module is responsible for performing a specific function such as user management, destination management, tour package handling, and booking management. This modular structure improves the system's scalability, maintainability, and clarity, making future updates and improvements easier.

During the implementation phase, each module was developed independently using modern web technologies. The frontend interface of the system is developed using HTML, CSS, and Bootstrap, which provides a responsive and user-friendly design. The backend logic is implemented using Python with the Django framework, which handles user requests, application logic, and data processing. The system uses SQLite as the database to store user information, tourist destinations, tour packages, and booking records.

System integration ensures that all modules communicate effectively with each other. During this stage, different components of the system were combined and tested to ensure smooth data flow between the frontend interface, backend server, and database. The integrated system allows tourists and administrators to interact with the platform efficiently, ensuring a reliable and seamless user experience.

3.1. User Authentication Module Snapshot and Coding

The User Authentication Module manages secure access to the Tourist Management System. It allows users to create an account, log in to the system, and securely access personalized travel features. This module ensures that only authenticated users can access booking services and other protected functionalities.

New users begin by registering through the Create Account page by providing essential details such as username, email address, and password. After submitting the registration form, the system stores user credentials securely in the database. Once registered, users can log in using their username and password through the login interface.

The authentication mechanism is implemented using Django's built-in authentication framework, which securely manages user sessions and password encryption. After successful login, users gain access to system features such as searching tourist destinations, viewing travel packages, and making bookings.

The system also prevents unauthorized access by restricting protected pages to authenticated users only. This ensures data security, user privacy, and controlled access to system functionalities.

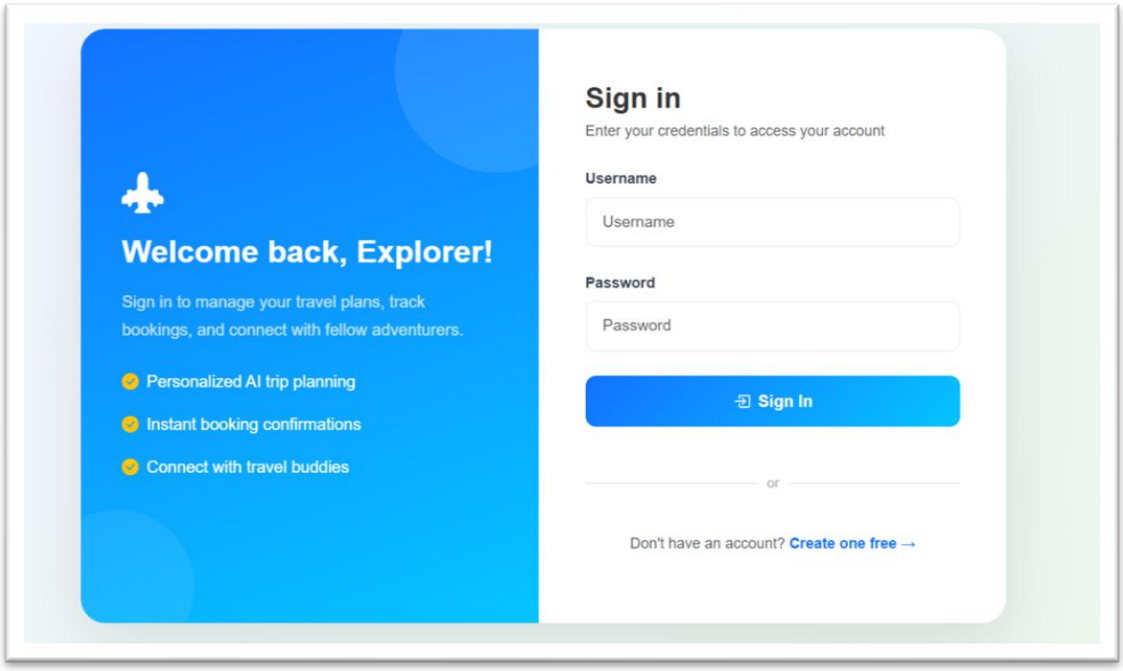

Fig 3.1. User Login

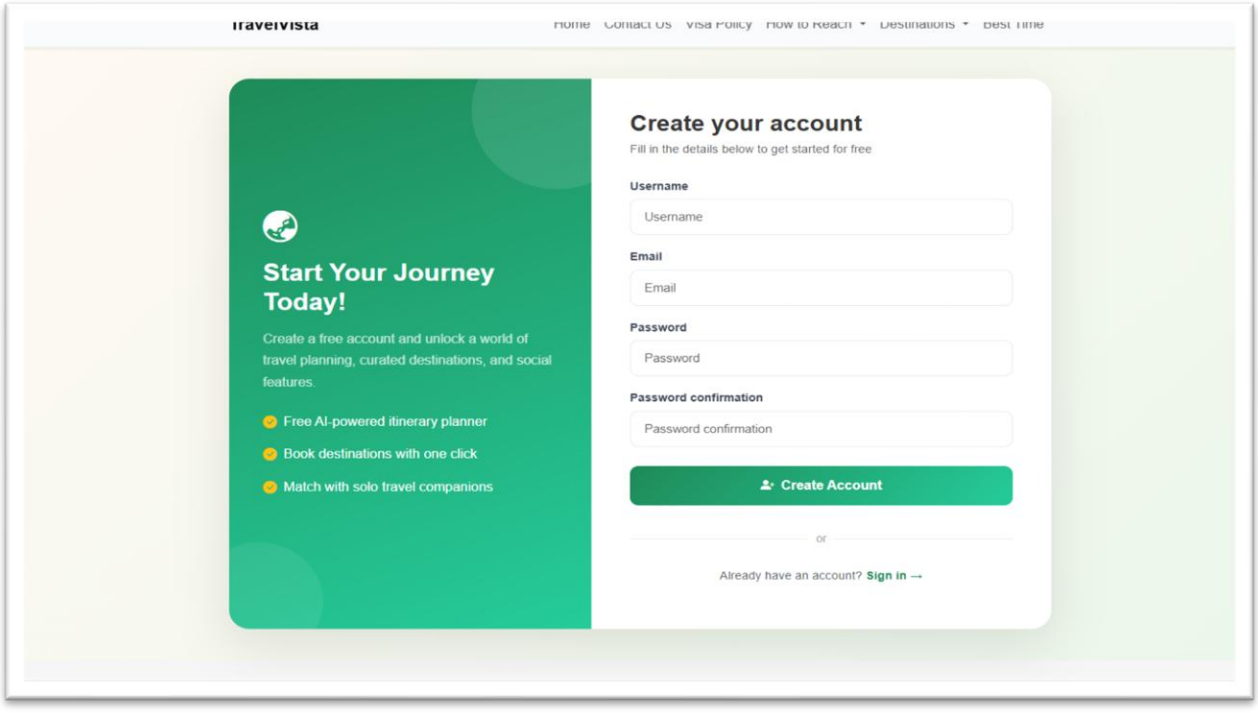

Fig 3.2. Registration Process

```
{% extends 'main/base.html' %}

{% block title %}Login – TravelVista{% endblock %}

{% block content %}
<style>
  /* Gradient background for the whole page */
  .auth-bg {
    background: linear-gradient(135deg, #f0f4ff 0%, #e8f5e9 100%);
    min-height: 85vh;
  }

  .auth-card {
    border-radius: 24px;
    box-shadow: 0 20px 60px rgba(0,0,0,0.1);
    overflow: hidden;
    animation: slideUp 0.6s ease-out;
    background: white;
  }

  @keyframes slideUp {
    from { opacity: 0; transform: translateY(40px); }
    to   { opacity: 1; transform: translateY(0); }
  }

  .auth-left {
    background: linear-gradient(160deg, #0d6efd 0%, #00c2ff 100%);
    position: relative;
    overflow: hidden;
  }

  .auth-left::before {
    content: '';
    position: absolute;
    top: -60px; right: -60px;
    width: 200px; height: 200px;
    border-radius: 50%;
    background: rgba(255,255,255,0.1);
  }

  .auth-left::after {
    content: '';
    position: absolute;
    bottom: -40px; left: -40px;
    width: 150px; height: 150px;
    border-radius: 50%;
    background: rgba(255,255,255,0.07);
  }

  .auth-form .form-control {
    border: 1.5px solid #e2e8f0;
    border-radius: 10px;
    padding: 12px 16px;
```

```
    font-size: 0.95rem;
    transition: all 0.3s ease;
  }

  .auth-form .form-control:focus {
    border-color: #0d6efd;
    box-shadow: 0 0 0 4px rgba(13,110,253,0.1);
  }

  .auth-btn {
    background: linear-gradient(135deg, #0d6efd, #00c2ff);
    border: none;
    border-radius: 10px;
    padding: 13px;
    font-weight: 700;
    color: white;
    transition: transform 0.2s ease;
  }

  .auth-btn:hover {
    opacity: 0.9;
    transform: translateY(-2px);
    color: white;
  }

  .form-field {
    animation: fadeField 0.5s ease forwards;
    opacity: 0;
  }
  .form-field:nth-child(1) { animation-delay: 0.1s; }
  .form-field:nth-child(2) { animation-delay: 0.2s; }

  @keyframes fadeField {
    from { opacity: 0; transform: translateX(-10px); }
    to   { opacity: 1; transform: translateX(0); }
  }
</style>

<div class="auth-bg d-flex align-items-center justify-content-center py-5">
  <div class="container">
    <div class="row justify-content-center">
      <div class="col-lg-10 col-xl-9">
        <div class="auth-card">
          <div class="row g-0">
            <!-- Left Panel (Hidden on below md) -->
            <div class="col-md-6 d-none d-md-flex flex-column justify-content-center p-5 auth-
                 left text-white">
              <div style="z-index: 2;">
                <i class="bi bi-airplane-engines-fill display-4 mb-4"></i>
                <h2 class="fw-bold mb-3">Welcome back!</h2>
                <p class="opacity-75 lead mb-4">
                  Sign in to manage your travel plans, track bookings, and connect with fellow
                  adventurers.
                </p>
```

```
                <div class="d-flex flex-column gap-3">
                  <div class="d-flex align-items-center gap-2">
                    <i class="bi bi-check-circle-fill text-warning"></i>
                    <span>Personalized AI trip planning</span>
                  </div>
                  <div class="d-flex align-items-center gap-2">
                    <i class="bi bi-check-circle-fill text-warning"></i>
                    <span>Instant booking confirmations</span>
                  </div>
                  <div class="d-flex align-items-center gap-2">
                    <i class="bi bi-check-circle-fill text-warning"></i>
                    <span>Connect with travel buddies</span>
                  </div>
                </div>
              </div>
            </div>

            <!-- Right Panel: Form -->
            <div class="col-md-6 p-4 p-md-5 bg-white">
              <div class="h-100 d-flex flex-column justify-content-center">
                <h3 class="fw-bold mb-2">Sign in</h3>
                <p class="text-muted mb-4 small">Enter your credentials to access your
                   account</p>

                <form method="POST" class="auth-form">
                  {% csrf_token %}
                  {% for field in form %}
                  <div class="mb-4 form-field">
                    <label for="{{ field.id_for_label }}" class="form-label fw-bold small text-
                   secondary mb-1">
                       {{ field.label }}
                    </label>
                    <input
                      type="{{ field.field.widget.input_type }}"
                      name="{{ field.html_name }}"
                      id="{{ field.id_for_label }}"
                      class="form-control {% if field.errors %}is-invalid{% endif %}"
                      {% if field.value %}value="{{ field.value }}"{% endif %}
                      placeholder="{{ field.label }}"
                      autocomplete="{{ field.html_name }}"
                    >
                    {% for error in field.errors %}
                      <div class="invalid-feedback">{{ error }}</div>
                    {% endfor %}
                  </div>
                  {% endfor %}

                  <div class="mb-3">
                    <button type="submit" class="btn auth-btn w-100">
                      <i class="bi bi-box-arrow-in-right me-2"></i>Sign In
                    </button>
                  </div>
                </form>
```

```
                    <div class="text-center mt-3">
                        <p class="text-muted small mb-0">
                            Don't have an account?
                            <a href="{% url 'register' %}" class="text-primary fw-bold text-decoration-
                        none">Create one free</a>
                        </p>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>

{% endblock %}
```

3.2. Home Page Module and Coding

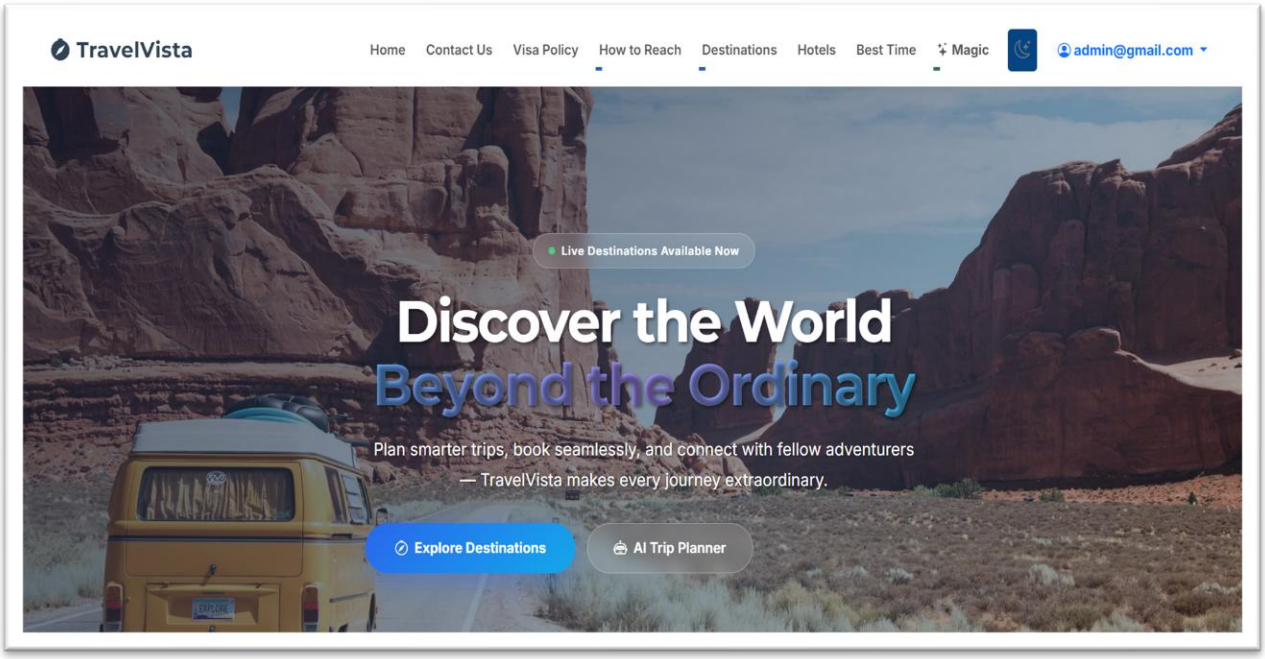

Fig 3.3. Home Page

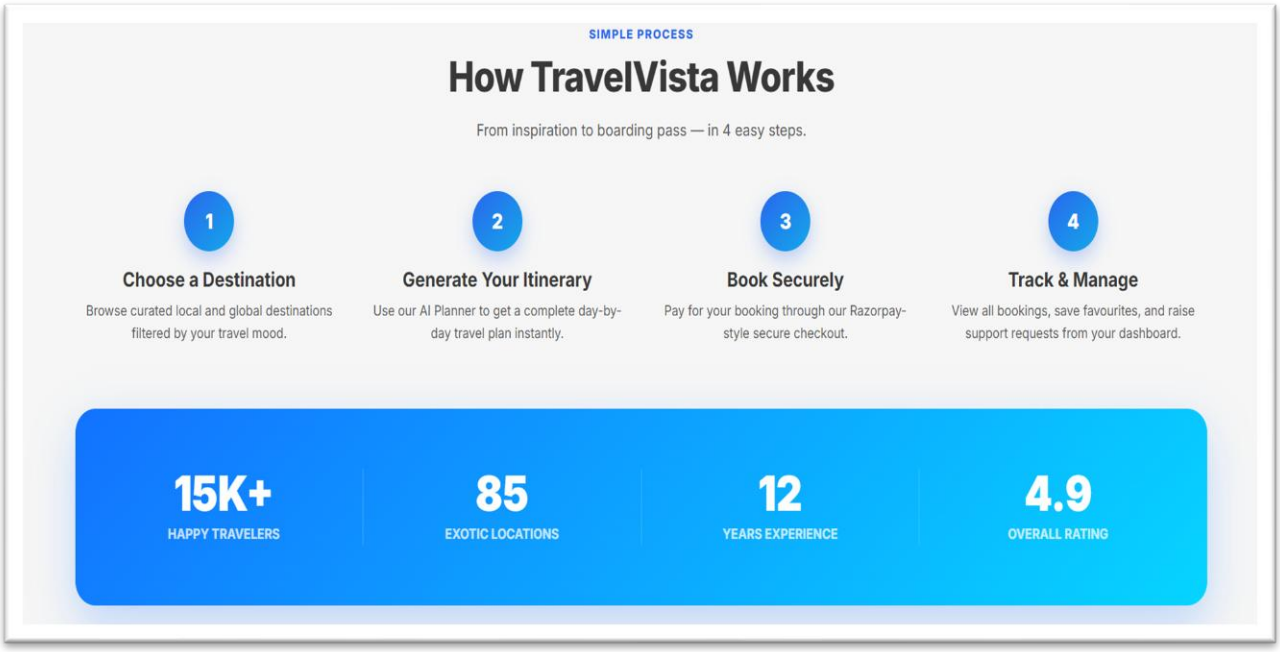

Fig 3.4. Working of TravelVista

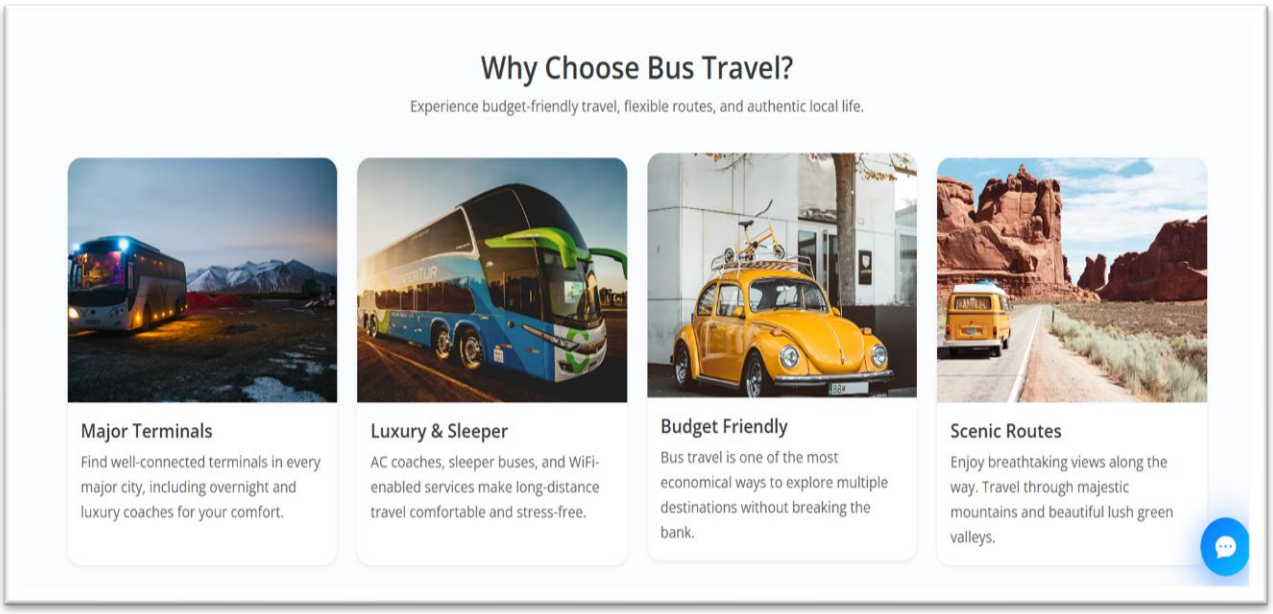

Fig 3.5. Destination Details Page

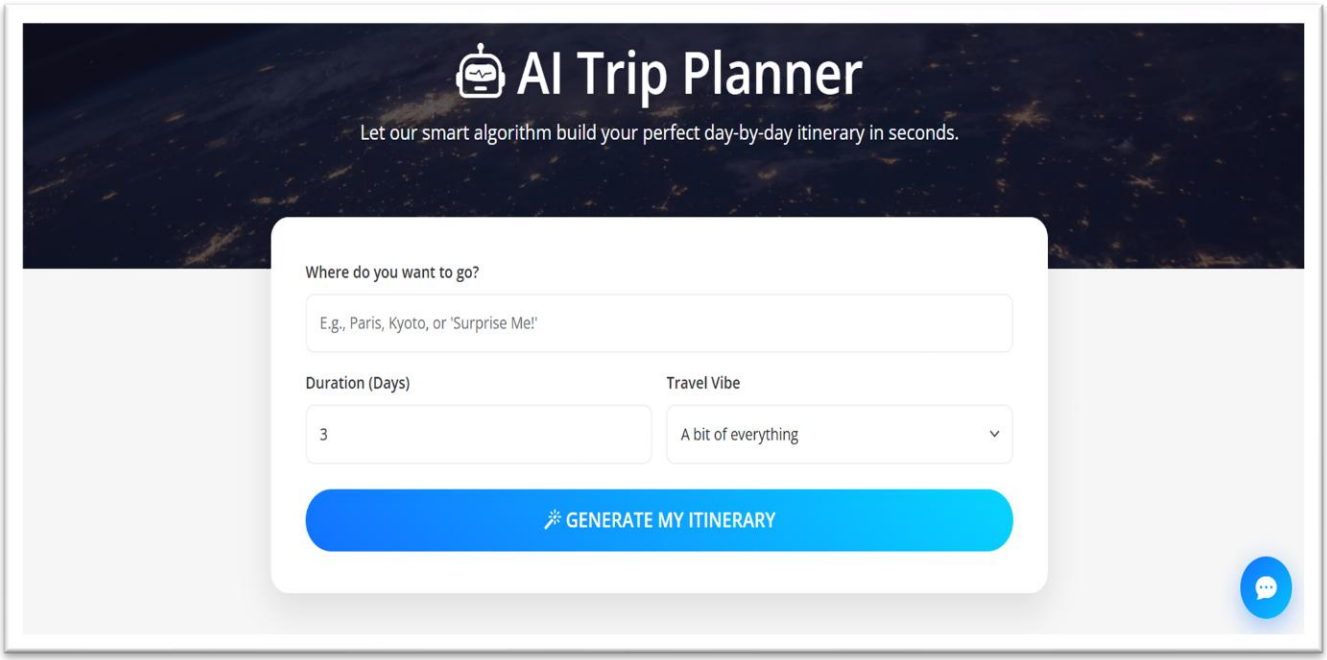

Fig 3.6. AI Trip Planner

```
<!-- ── HERO ── -->
<section class="hero-section">
  <div class="hero-overlay"></div>
  <div class="container hero-inner ps-4 ps-md-5">
    <div class="hero-pill">
      <span class="dot"></span> Live Destinations Available Now
    </div>
    <h1 class="hero-title">
      Discover the World<br>
      <span>Beyond the Ordinary</span>
    </h1>
    <p class="hero-sub">
      Plan smarter trips, book seamlessly, and connect with fellow adventurers — TravelVista makes
every journey
      extraordinary.
    </p>
    <div class="hero-btns">
      <a href="{% url 'destination_list' %}" class="btn-hero-primary">
        <i class="bi bi-compass me-2"></i>Explore Destinations
      </a>
      <a href="{% url 'ai_planner' %}" class="btn-hero-ghost">
        <i class="bi bi-robot me-2"></i>AI Trip Planner
      </a>
    </div>
  </div>
  <!-- Search Bar floating at bottom of hero -->
  <div class="hero-search-bar">
    <div class="container">
      <form action="{% url 'destination_list' %}" method="GET" class="search-card">
        <div class="search-field">
          <label><i class="bi bi-geo-alt me-1"></i>Destination</label>
          <input type="text" name="q" placeholder="Where do you want to go?">
        </div>
        <div class="search-field">
          <label><i class="bi bi-calendar me-1"></i>Travel Date</label>
          <input type="date">
        </div>
        <div class="search-field">
          <label><i class="bi bi-people me-1"></i>Travellers</label>
          <select>
            <option>1 Person</option>
            <option>2 People</option>
            <option>3–5 People</option>
            <option>Group (6+)</option>
          </select>
        </div>
        <button type="submit" class="search-btn">
          <i class="bi bi-search me-2"></i>Search
        </button>
      </form>
    </div>
  </div>
```

```
</section>

<!-- SEARCH BAR SPACER -->
<div style="padding-top: 80px; background: #f8fafc;"></div>

<!-- ── TRAVEL CATEGORIES ── -->
<section class="py-5 bg-light fade-in">
  <div class="container">
    <div class="text-center mb-5">
      <div class="section-kicker">Browse by Mood</div>
      <h2 class="section-heading">What kind of trip?</h2>
    </div>
    <div class="row g-3">
      <div class="col-6 col-md-3 col-lg-2">
        <a href="{% url 'destination_list' %}" class="cat-card">
          <div class="cat-icon" style="background:#dbeafe; color:#2563eb;">🏖️</div>
          <div class="fw-700 small text-dark fw-bold">Beach</div>
        </a>
      </div>
      <div class="col-6 col-md-3 col-lg-2">
        <a href="{% url 'destination_list' %}" class="cat-card">
          <div class="cat-icon" style="background:#dcfce7; color:#16a34a;">⛰️</div>
          <div class="fw-700 small text-dark fw-bold">Mountains</div>
        </a>
      </div>
      <div class="col-6 col-md-3 col-lg-2">
        <a href="{% url 'destination_list' %}" class="cat-card">
          <div class="cat-icon" style="background:#fef3c7; color:#d97706;">🏛️</div>
          <div class="fw-700 small text-dark fw-bold">Heritage</div>
        </a>
      </div>
      <div class="col-6 col-md-3 col-lg-2">
        <a href="{% url 'mood_filter' %}" class="cat-card">
          <div class="cat-icon" style="background:#fce7f3; color:#db2777;">🌸</div>
          <div class="fw-700 small text-dark fw-bold">Romantic</div>
        </a>
      </div>
      <div class="col-6 col-md-3 col-lg-2">
        <a href="{% url 'destination_list' %}" class="cat-card">
          <div class="cat-icon" style="background:#ede9fe; color:#7c3aed;">🌆</div>
          <div class="fw-700 small text-dark fw-bold">City Break</div>
        </a>
      </div>
      <div class="col-6 col-md-3 col-lg-2">
        <a href="{% url 'mood_filter' %}" class="cat-card">
          <div class="cat-icon" style="background:#ffedd5; color:#ea580c;">🔥</div>
          <div class="fw-700 small text-dark fw-bold">Adventure</div>
        </a>
      </div>
    </div>
  </div>
</section>
```

```
<!-- ── WHY CHOOSE US ── -->
<section class="py-5 mt-2 fade-in">
  <div class="container">
    <div class="text-center mb-5">
      <div class="section-kicker">Why TravelVista?</div>
      <h2 class="section-heading">Redefining the Way You Travel</h2>
      <p class="text-muted mt-3 mx-auto" style="max-width: 600px;">We take the stress out of planning
so you can
        focus on the experience.</p>
    </div>
    <div class="row g-4">
      <div class="col-md-4">
        <div class="feature-card h-100 position-relative">
          <div class="feature-icon"><i class="bi bi-stars"></i></div>
          <div class="position-absolute top-0 end-0 p-3 opacity-25 sparkle-container"><i class="bi bi-
stars"
              style="font-size: 2rem;"></i></div>
          <h4 class="fw-bold mb-2">AI-Powered Planning</h4>
          <p class="text-muted mb-0">Generate full day-by-day itineraries tailored to your travel vibe
and
            duration in just one click.</p>
        </div>
      </div>
      <div class="col-md-4">
        <div class="feature-card h-100">
          <div class="feature-icon"><i class="bi bi-shield-check"></i></div>
          <h4 class="fw-bold mb-2">Seamless Bookings</h4>
          <p class="text-muted mb-0">Integrated payments, transparent pricing, and instant
confirmations for
            all your reservations.</p>
        </div>
      </div>
      <div class="col-md-4">
        <div class="feature-card h-100 position-relative">
          <div class="feature-icon"><i class="bi bi-people-fill"></i></div>
          <div class="position-absolute top-0 end-0 p-3 opacity-25 sparkle-container"><i
              class="bi bi-person-heart" style="font-size: 2rem;"></i></div>
          <h4 class="fw-bold mb-2">Buddy Matcher</h4>
          <p class="text-muted mb-0">Don't want to travel solo? Connect with verified travelers
heading to the
            same destinations.</p>
        </div>
      </div>
    </div>
  </div>
</section>

<!-- ── TRENDING ESCAPES ── -->
<section class="py-5 bg-light fade-in">
  <div class="container">
    <div class="d-flex justify-content-between align-items-end mb-5">
      <div>
        <div class="section-kicker">Inspiration</div>
        <h2 class="section-heading mb-0">Trending Escapes</h2>
```

```
        </div>
        <a href="{% url 'destination_list' %}" class="btn btn-outline-primary rounded-pill px-4">See All
<i
                class="bi bi-arrow-right"></i></a>
      </div>
      <div class="row g-4">
        {% for dest in featured %}
        <div
          class="{% if forloop.first %}col-lg-6{% elif forloop.counter <= 3 %}col-lg-3 col-md-6{% else
%}col-lg-4 col-md-6{% endif %}">
          <a href="{% url 'book_destination' dest.id %}" class="text-decoration-none">
            <div class="card trending-card">
              {% if dest.image %}
              <img src="{{ dest.image.url }}" alt="{{ dest.name }}">
              {% else %}
              <img src="https://images.unsplash.com/photo-1469854523086-cc02fe5d8800" alt="{{
dest.name }}">
              {% endif %}
              <div class="trending-overlay">
                {% if dest.is_featured %}
                <span class="badge bg-danger mb-2 px-3 py-2 rounded-pill"><i class="bi bi-fire"></i>
                  Featured</span>
                {% endif %}
                <h3 class="fw-bold mb-1">{{ dest.name }}</h3>
                <p class="mb-0 text-light opacity-75">{{ dest.category }} &middot; ₹{{ dest.price
}}</p>
              </div>
            </div>
          </a>
        </div>
        {% empty %}
        <div class="col-12 text-center py-5">
          <p class="text-muted">Stay tuned! We're hand-picking the best escapes for you.</p>
        </div>
        {% endfor %}
      </div>
    </div>
</section>

<!-- ── HOW IT WORKS ── -->
<section class="py-5 fade-in">
  <div class="container">
    <div class="text-center mb-5">
      <div class="section-kicker">Simple Process</div>
      <h2 class="section-heading">How TravelVista Works</h2>
      <p class="text-muted mt-3 mx-auto" style="max-width: 540px;">From inspiration to boarding pass
— in 4 easy
        steps.</p>
    </div>
    <div class="row g-4 text-center">
      <div class="col-6 col-md-3">
        <div class="step-circle">1</div>
        <h5 class="fw-bold mb-2">Choose a Destination</h5>
        <p class="text-muted small">Browse curated local and global destinations filtered by your travel
```

```
mood.
            </p>
          </div>
          <div class="col-6 col-md-3">
            <div class="step-circle">2</div>
            <h5 class="fw-bold mb-2">Generate Your Itinerary</h5>
            <p class="text-muted small">Use our AI Planner to get a complete day-by-day travel plan
instantly.</p>
          </div>
          <div class="col-6 col-md-3">
            <div class="step-circle">3</div>
            <h5 class="fw-bold mb-2">Book Securely</h5>
            <p class="text-muted small">Pay for your booking through our Razorpay-style secure
checkout.</p>
          </div>
          <div class="col-6 col-md-3">
            <div class="step-circle">4</div>
            <h5 class="fw-bold mb-2">Track & Manage</h5>
            <p class="text-muted small">View all bookings, save favourites, and raise support requests from
your
              dashboard.</p>
          </div>
        </div>
      </div>
    </section>

    <!-- ── STATS ── -->
    <section class="container fade-in">
      <div class="stats-section">
        <div class="row text-center px-4">
          <div class="col-md-3 col-6 mb-4 mb-md-0 border-end border-light border-opacity-25">
            <div class="stat-number">15K+</div>
            <div class="fw-bold text-uppercase opacity-75 small mt-1">Happy Travelers</div>
          </div>
          <div class="col-md-3 col-6 mb-4 mb-md-0 border-end border-light border-opacity-25">
            <div class="stat-number">85</div>
            <div class="fw-bold text-uppercase opacity-75 small mt-1">Exotic Locations</div>
          </div>
          <div class="col-md-3 col-6 border-end border-light border-opacity-25">
            <div class="stat-number">12</div>
            <div class="fw-bold text-uppercase opacity-75 small mt-1">Years Experience</div>
          </div>
          <div class="col-md-3 col-6">
            <div class="stat-number d-flex justify-content-center align-items-center gap-2">
              4.9 <i class="bi bi-star-fill text-warning" style="font-size: 2rem;"></i>
            </div>
            <div class="fw-bold text-uppercase opacity-75 small mt-1">Overall Rating</div>
          </div>
        </div>
      </div>
    </section>

    <!-- ── TESTIMONIALS ── -->
    <section class="py-5 mt-3 fade-in">
```

```
<div class="container">
  <div class="text-center mb-5">
    <div class="section-kicker">Real Reviews</div>
    <h2 class="section-heading">What Our Travelers Say</h2>
  </div>
  <div class="row g-4">
    <div class="col-md-4">
      <div class="testimonial-card h-100">
        <div class="quote-icon">"</div>
        <div class="stars mb-3">★★★★★</div>
        <p class="text-muted mb-4">"TravelVista completely changed how I plan my trips. The AI planner gave
          me a 7-day itinerary for Rajasthan in under 30 seconds!"</p>
        <div class="d-flex align-items-center gap-10">
          <div class="rounded-circle bg-primary text-white d-flex align-items-center justify-content-center fw-bold me-3"
            style="width:44px; height:44px; font-size:1.1rem;">A</div>
          <div>
            <div class="fw-bold text-dark">Arjun Sharma</div>
            <div class="text-muted small">Mumbai, India</div>
          </div>
        </div>
      </div>
    </div>
    <div class="col-md-4">
      <div class="testimonial-card h-100">
        <div class="quote-icon">"</div>
        <div class="stars mb-3">★★★★★</div>
        <p class="text-muted mb-4">"Booked a trip to Munnar through TravelVista. The checkout was smooth,
          got a confirmation instantly, and the dashboard made tracking super easy."</p>
        <div class="d-flex align-items-center">
          <div class="rounded-circle bg-success text-white d-flex align-items-center justify-content-center fw-bold me-3"
            style="width:44px; height:44px; font-size:1.1rem;">P</div>
          <div>
            <div class="fw-bold text-dark">Priya Nair</div>
            <div class="text-muted small">Bangalore, India</div>
          </div>
        </div>
      </div>
    </div>
    <div class="col-md-4">
      <div class="testimonial-card h-100">
        <div class="quote-icon">"</div>
        <div class="stars mb-3">★★★★☆</div>
        <p class="text-muted mb-4">"The buddy matcher helped me find a fellow traveler for my solo trip to
          Ladakh. Absolutely love this platform — can't imagine booking trips any other way!"</p>
        <div class="d-flex align-items-center">
          <div class="rounded-circle bg-warning text-dark d-flex align-items-center justify-content-center fw-bold me-3"
            style="width:44px; height:44px; font-size:1.1rem;">R</div>
```

```
                <div>
                  <div class="fw-bold text-dark">Rahul Mehta</div>
                  <div class="text-muted small">Delhi, India</div>
                </div>
              </div>
            </div>
          </div>
        </div>
      </section>

<!-- ── CTA BANNER ── -->
<section class="container pb-5 fade-in">
  <div class="cta-banner">
    <div style="position: relative; z-index: 1;" class="row align-items-center">
      <div class="col-lg-8 mb-4 mb-lg-0">
        <div class="section-kicker" style="color: #93c5fd;">Start Today</div>
        <h2 class="section-heading text-white mb-3">Ready for Your Next Adventure?</h2>
        <p style="color: rgba(255,255,255,0.7); font-size: 1.05rem; line-height: 1.75; max-
width:540px;">
          Join thousands of smart travelers who plan better, book faster, and explore more with
TravelVista.
          It's completely free to get started.
        </p>
      </div>
      <div class="col-lg-4 text-lg-end d-flex flex-column align-items-lg-end gap-3">
        {% if user.is_authenticated %}
        <a href="{% url 'user_dashboard' %}" class="btn-hero-primary" style="text-align:center;">
          <i class="bi bi-speedometer2 me-2"></i>View Your Dashboard
        </a>
        {% else %}
        <a href="{% url 'register' %}" class="btn-hero-primary" style="text-align:center;">
          <i class="bi bi-person-plus-fill me-2"></i>Create Free Account
        </a>
        {% endif %}
        <a href="{% url 'destination_list' %}" class="btn-hero-ghost d-block text-center">
          <i class="bi bi-compass me-2"></i>Browse Destinations
        </a>
      </div>
    </div>
  </div>
</section>

<script>
  // Smooth fade-in on scroll
  const observer = new IntersectionObserver(entries => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        entry.target.classList.add('visible');
        observer.unobserve(entry.target);
      }
    });
  }, { threshold: 0.1 });
```

```
    document.querySelectorAll('.fade-in').forEach(el => observer.observe(el));

    // Animate stat counters
    function animateCounters() {
        document.querySelectorAll('.stat-number').forEach(el => {
            const text = el.innerText;
            const num = parseFloat(text.replace(/[^0-9.]/g, ''));
            if (isNaN(num)) return;
            let start = 0;
            const suffix = text.replace(/[0-9.]/g, '');
            const duration = 1800;
            const step = num / (duration / 16);
            const timer = setInterval(() => {
                start += step;
                if (start >= num) { start = num; clearInterval(timer); }
                el.innerText = (Number.isInteger(num) ? Math.floor(start) : start.toFixed(1)) + suffix;
            }, 16);
        });
    }

    // Trigger counters when stats section is visible
    const statsSection = document.querySelector('.stats-section');
    if (statsSection) {
        const statsObs = new IntersectionObserver(entries => {
            if (entries[0].isIntersecting) { animateCounters(); statsObs.disconnect(); }
        }, { threshold: 0.3 });
        statsObs.observe(statsSection);
    }
</script>

{% endblock %}
```

3.3. Hotel Search and Filtering Module Snapshots and Coding

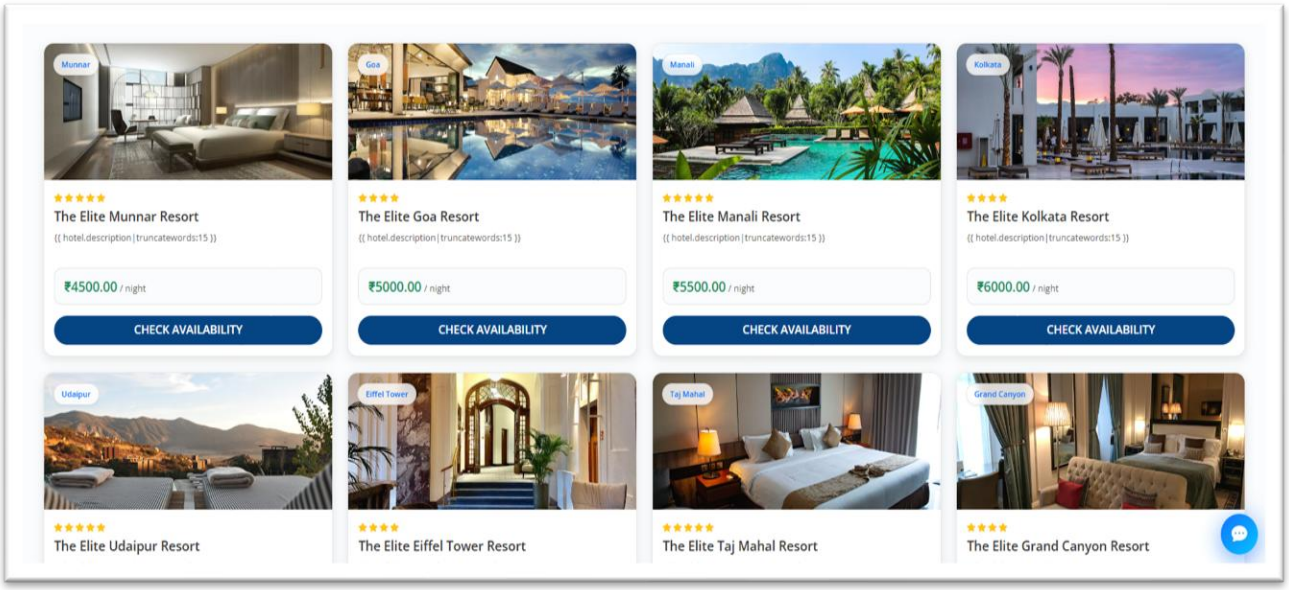

Fig 3.7. Available Hotels

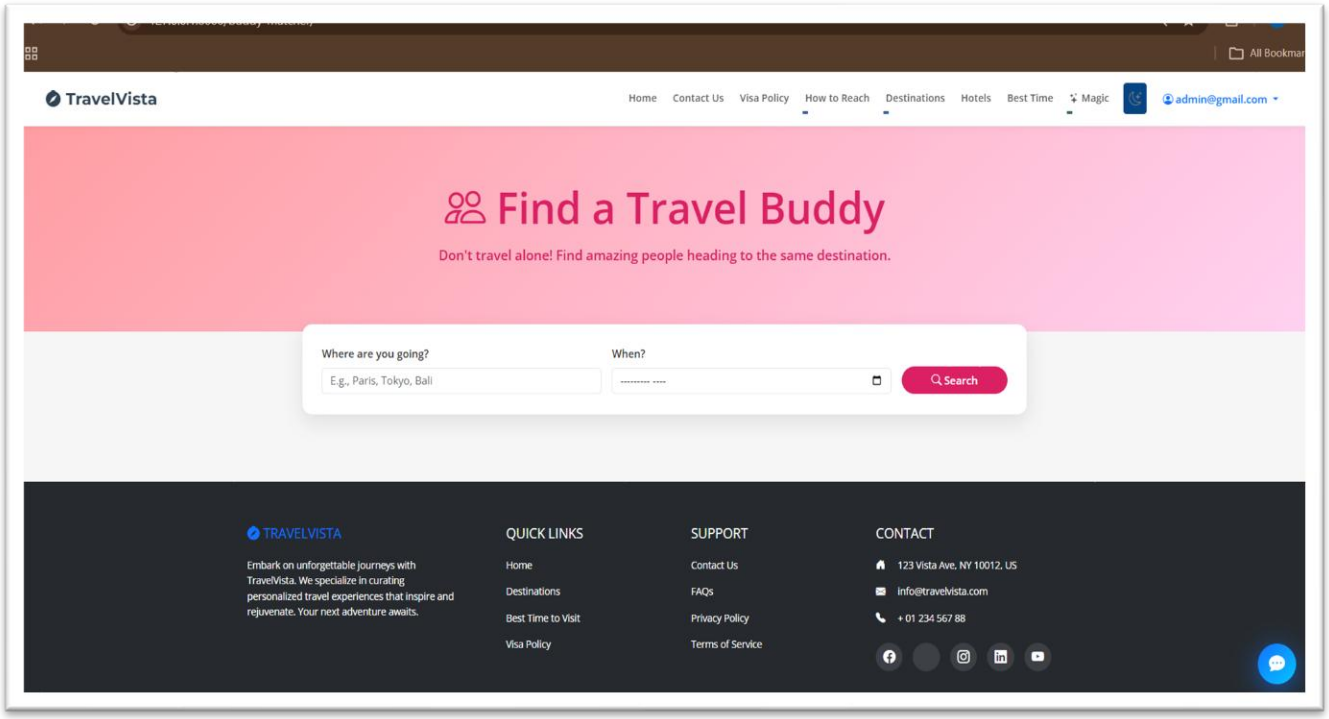

Fig 3.8. Travel Togather

```
{% extends 'main/base.html' %}

{% block title %}Book {{ hotel.name }} – TravelVista{% endblock %}

{% block content %}
<div class="container py-5 mt-4 fade-in">
  <div class="row g-5">
    <div class="col-lg-8">
      <div class="card border-0 shadow-lg rounded-4 overflow-hidden mb-4">
        {% if hotel.image %}
        <img src="{{ hotel.image.url }}" class="img-fluid"
          style="height: 450px; object-fit: cover; width:100%;">
        {% else %}
        <img src="https://images.unsplash.com/photo-1542314841-3827178a1efb?q=80&w=1200"
class="img-fluid"
          style="height: 450px; object-fit: cover; width:100%;">
        {% endif %}
      </div>
      <h1 class="fw-bold mb-3">{{ hotel.name }}</h1>
      <p class="text-muted mb-4"><i class="bi bi-geo-alt-fill text-primary me-2"></i>{{
hotel.destination.name }}
      </p>
      <div class="mb-5">
        <h4 class="fw-bold mb-3">About this hotel</h4>
        <p class="text-muted" style="line-height: 1.8;">{{ hotel.description }}</p>
      </div>
      <div class="mb-5">
        <h4 class="fw-bold mb-3">Amenities</h4>
        <div class="d-flex flex-wrap gap-2">
          {% for amenity in hotel.amenities.split %}
          <span class="badge bg-light text-dark border rounded-pill">{{ amenity|cut:"," }}</span>
          {% endfor %}
        </div>
      </div>
    </div>

    <div class="col-lg-4">
      <div class="card border-0 shadow-sm rounded-4 sticky-top" style="top: 100px;">
        <div class="card-body p-4">
          <h5 class="fw-bold mb-4">Book Your Stay</h5>
          <form method="POST">
            {% csrf_token %}
            <div class="mb-3">
              <label class="form-label small fw-bold text-muted">CHECK-IN</label>
              <input type="date" name="check_in" id="check_in" class="form-control" required>
            </div>
            <div class="mb-3">
              <label class="form-label small fw-bold text-muted">CHECK-OUT</label>
              <input type="date" name="check_out" id="check_out" class="form-control" required>
            </div>
            <div class="mb-4">
              <label class="form-label small fw-bold text-muted">GUESTS</label>
              <select name="guests" id="guests" class="form-select">
```

```
                        <option value="1">1 Guest</option>
                        <option value="2">2 Guests</option>
                        <option value="3">3 Guests</option>
                    </select>
                </div>
                <div class="border-top pt-3 mb-4">
                    <div class="d-flex justify-content-between mb-2">
                        <span class="text-muted">₹{{ hotel.price_per_night }} x <span
id="night_count">1</span>
                            night(s)</span>
                        <span class="fw-bold" id="total_display">₹{{ hotel.price_per_night }}</span>
                    </div>
                    <div class="d-flex justify-content-between">
                        <span class="text-muted">Total (incl. GST)</span>
                        <h4 class="fw-bold text-success mb-0" id="final_total">₹{{ hotel.price_per_night
}}</h4>
                    </div>
                </div>
                <button type="submit" class="btn btn-primary w-100 rounded-pill py-3 fw-bold shadow">
                    Reserve Room
                </button>
            </form>
        </div>
    </div>
</div>
</div>

<script>
    const checkIn = document.getElementById('check_in');
    const checkOut = document.getElementById('check_out');
    const guests = document.getElementById('guests');
    const nightCount = document.getElementById('night_count');
    const totalDisplay = document.getElementById('total_display');
    const finalTotal = document.getElementById('final_total');

    const pricePerNight = parseFloat("{{ hotel.price_per_night }}");

    function updatePrice() {
        if (checkIn.value && checkOut.value) {
            const start = new Date(checkIn.value);
            const end = new Date(checkOut.value);
            const diff = (end - start) / (1000 * 60 * 60 * 24);
            const nights = Math.max(1, Math.ceil(diff));
            const guestCount = parseInt(guests.value);

            const total = pricePerNight * nights * guestCount;

            nightCount.innerText = nights;
            totalDisplay.innerText = '₹' + (pricePerNight * nights).toLocaleString();
            finalTotal.innerText = '₹' + total.toLocaleString();
        }
    }

    checkIn.addEventListener('change', updatePrice);
```

```
    checkOut.addEventListener('change', updatePrice);
    guests.addEventListener('change', updatePrice);

    // Set min dates to today
    const today = new Date().toISOString().split('T')[0];
    checkIn.min = today;
    checkIn.addEventListener('change', () => {
        checkOut.min = checkIn.value;
    });
</script>
{% endblock %}
```

3.4. Booking and Payment Module Snapshots and Coding

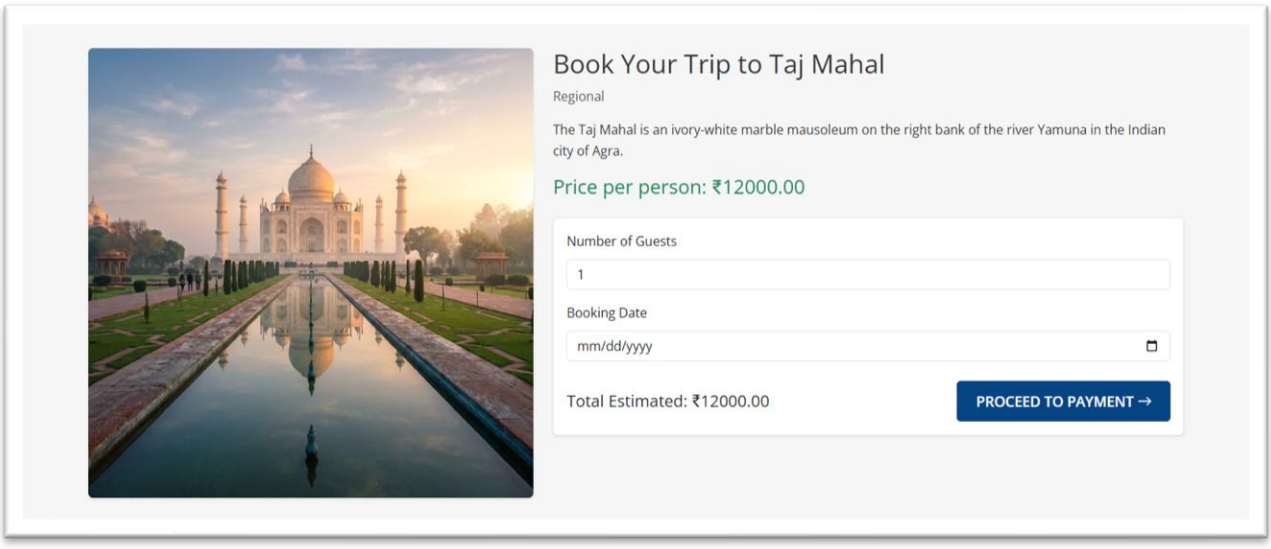

Fig 3.9. Booking Page

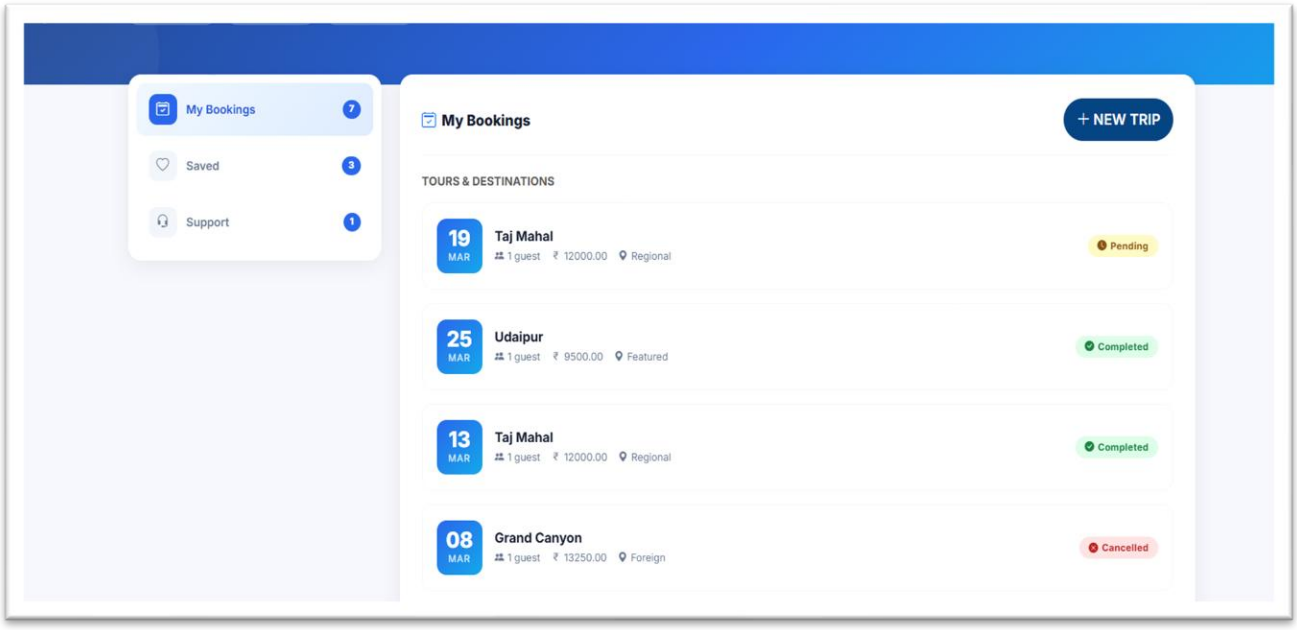

Fig 3.10. Booking Status

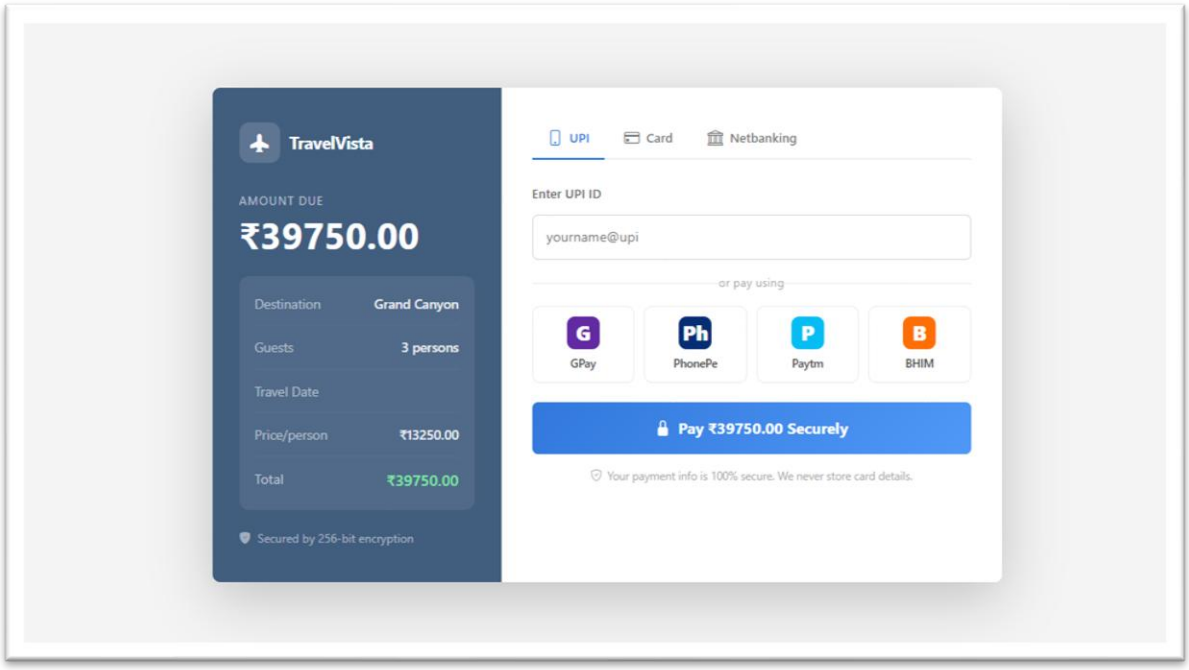

Fig 3.11. Payment Page

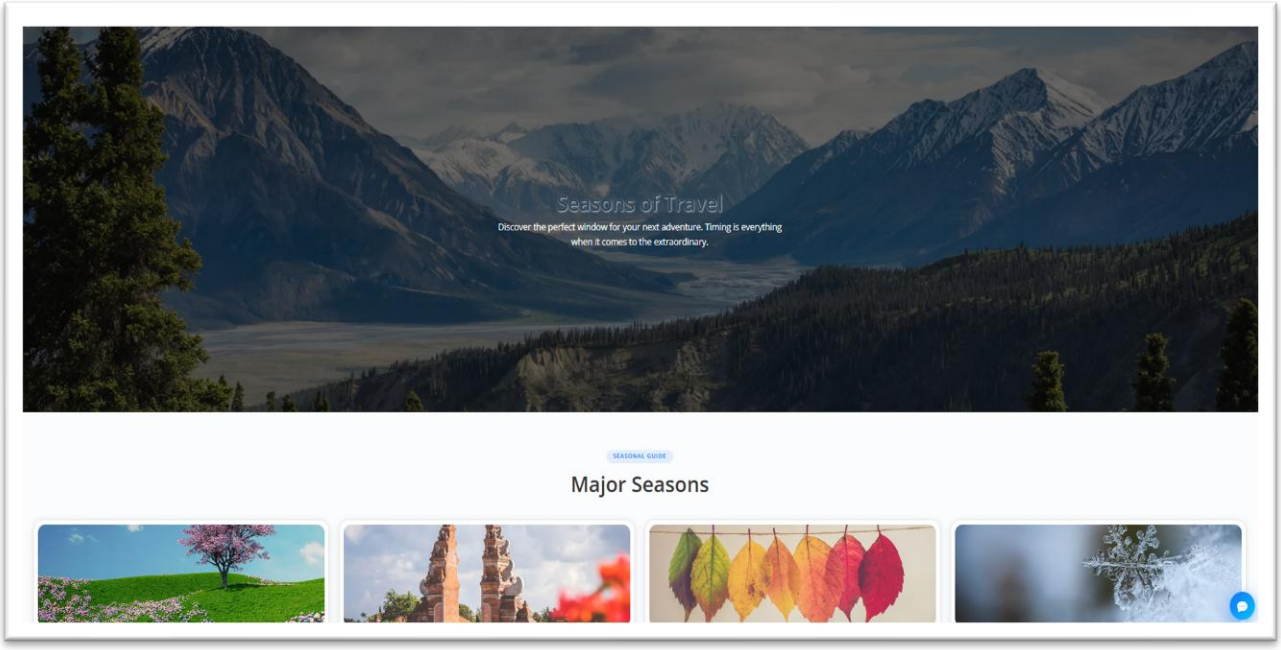

Fig 3.12. Season for Travel

```
<div class="rz-page">
  <div class="rz-modal">
    <!-- Left Brand Panel -->
    <div class="rz-left">
      <div class="rz-brand">
        <div class="rz-brand-icon"><i class="bi bi-airplane-fill"></i></div>
        <span class="rz-brand-name">TravelVista</span>
      </div>

      <div class="rz-amount-label">Amount Due</div>
      <div class="rz-amount">₹{{ booking.total_price }}</div>

      <div class="rz-booking-detail">
        <div class="detail-row">
          <span>Destination</span>
          <span>{{ booking.destination.name }}</span>
        </div>
        <hr class="rz-divider-line">
        <div class="detail-row">
          <span>Guests</span>
          <span>{{ booking.guests }} person{% if booking.guests > 1 %}s{% endif %}</span>
        </div>
        <hr class="rz-divider-line">
        <div class="detail-row">
          <span>Travel Date</span>
          <span>{{ booking.booking_date|date:"d M Y" }}</span>
        </div>
        <hr class="rz-divider-line">
        <div class="detail-row">
          <span>Price/person</span>
          <span>₹{{ booking.destination.price }}</span>
        </div>
        <hr class="rz-divider-line">
        <div class="detail-row" style="color:#fff; font-size:0.9rem;">
          <span style="color:rgba(255,255,255,0.6)">Total</span>
          <span style="color:#7ee8a2; font-size:1rem;">₹{{ booking.total_price }}</span>
        </div>
      </div>

      <div class="rz-secure-tag">
        <i class="bi bi-shield-fill-check"></i>
        Secured by 256-bit encryption
      </div>
    </div>

    <!-- Right Form Panel -->
    <div class="rz-right">
      <div class="rz-tabs">
        <div class="rz-tab active" onclick="switchTab('upi', this)">
          <i class="bi bi-phone"></i> UPI
        </div>
        <div class="rz-tab" onclick="switchTab('card', this)">
          <i class="bi bi-credit-card"></i> Card
        </div>
```

```
            <div class="rz-tab" onclick="switchTab('netbanking', this)">
              <i class="bi bi-bank"></i> Netbanking
            </div>
          </div>

          <!-- UPI Section -->
          <div class="rz-section active" id="sec-upi">
            <div style="font-size:0.82rem; color:#666; margin-bottom:12px; font-weight:600;">Enter
UPI ID</div>
            <div class="rz-upi-input-row">
              <input type="text" placeholder="yourname@upi" id="upi-field">
            </div>

            <div class="rz-or-divider">or pay using</div>

            <div class="rz-upi-apps">
              <div class="rz-upi-app" onclick="selectUpi(this)">
                <div class="upi-app-icon" style="background:#5f259f; color:#fff;">G</div>
                GPay
              </div>
              <div class="rz-upi-app" onclick="selectUpi(this)">
                <div class="upi-app-icon" style="background:#002970; color:#fff;">Ph</div>
                PhonePe
              </div>
              <div class="rz-upi-app" onclick="selectUpi(this)">
                <div class="upi-app-icon" style="background:#00baf2; color:#fff;">P</div>
                Paytm
              </div>
              <div class="rz-upi-app" onclick="selectUpi(this)">
                <div class="upi-app-icon" style="background:#ff6b00; color:#fff;">B</div>
                BHIM
              </div>
            </div>

            <form method="POST" action="{% url 'process_payment' booking.id %}">
              {% csrf_token %}
              <button type="submit" class="rz-pay-btn">
                <i class="bi bi-lock-fill"></i> Pay ₹{{ booking.total_price }} Securely
              </button>
            </form>
          </div>

          <!-- Card Section -->
          <div class="rz-section" id="sec-card">
            <div class="rz-card-icons">
              <span class="rz-card-badge" style="color:#1a1f71;">VISA</span>
              <span class="rz-card-badge" style="color:#eb001b;">MC</span>
              <span class="rz-card-badge" style="color:#003087;">AMEX</span>
              <span class="rz-card-badge" style="color:#ff6600;">Rupay</span>
            </div>

            <form method="POST" action="{% url 'process_payment' booking.id %}">
              {% csrf_token %}
              <div class="rz-field">
                <label>Cardholder Name</label>
```

```
            <input type="text" placeholder="As on card">
          </div>
          <div class="rz-field">
            <label>Card Number</label>
            <input type="text" placeholder="1234  5678  9012  3456" maxlength="19"
              oninput="formatCard(this)">
          </div>
          <div class="rz-field-row">
            <div class="rz-field">
              <label>Expiry</label>
              <input type="text" placeholder="MM / YY" maxlength="5">
            </div>
            <div class="rz-field">
              <label>CVV</label>
              <input type="password" placeholder="• • •" maxlength="4">
            </div>
          </div>
          <button type="submit" class="rz-pay-btn">
            <i class="bi bi-lock-fill"></i> Pay ₹{{ booking.total_price }} Securely
          </button>
        </form>
      </div>

      <!-- Netbanking Section -->
      <div class="rz-section" id="sec-netbanking">
          <div style="font-size:0.82rem; color:#666; margin-bottom:12px; font-
weight:600;">Select your bank</div>
        <div class="rz-bank-list">
          <div class="rz-bank-item" onclick="selectBank(this)">
            <div class="rz-bank-icon" style="background:#0066b2;">SBI</div>
            State Bank
          </div>
          <div class="rz-bank-item" onclick="selectBank(this)">
            <div class="rz-bank-icon" style="background:#e03f2a;">HDFC</div>
            HDFC Bank
          </div>
          <div class="rz-bank-item" onclick="selectBank(this)">
            <div class="rz-bank-icon" style="background:#f7941d;">ICIC</div>
            ICICI Bank
          </div>
          <div class="rz-bank-item" onclick="selectBank(this)">
            <div class="rz-bank-icon" style="background:#c00;">AXIS</div>
            Axis Bank
          </div>
          <div class="rz-bank-item" onclick="selectBank(this)">
            <div class="rz-bank-icon" style="background:#1e4799;">PNB</div>
            Punjab Natl.
          </div>
          <div class="rz-bank-item" onclick="selectBank(this)">
            <div class="rz-bank-icon" style="background:#4caf50;">BOI</div>
            Bank of India
          </div>
        </div>

        <form method="POST" action="{% url 'process_payment' booking.id %}">
```

```
 {% csrf_token %}
                <button type="submit" class="rz-pay-btn">
                   <i class="bi bi-lock-fill"></i> Proceed to Bank
                </button>
             </form>
          </div>

          <div class="rz-footer-note">
             <i class="bi bi-shield-check"></i>
             Your payment info is 100% secure. We never store card details.
          </div>
       </div>
    </div>
</div>

<script>
    function switchTab(name, el) {
       document.querySelectorAll('.rz-tab').forEach(t => t.classList.remove('active'));
       document.querySelectorAll('.rz-section').forEach(s => s.classList.remove('active'));
       el.classList.add('active');
       document.getElementById('sec-' + name).classList.add('active');
    }

    function selectUpi(el) {
       document.querySelectorAll('.rz-upi-app').forEach(a => {
          a.style.borderColor = '#e8e8e8';
          a.style.background = '';
       });
       el.style.borderColor = '#2d74da';
       el.style.background = '#f0f6ff';
    }

    function selectBank(el) {
       document.querySelectorAll('.rz-bank-item').forEach(b => {
          b.style.borderColor = '#e8e8e8';
          b.style.background = '';
       });
       el.style.borderColor = '#2d74da';
       el.style.background = '#f0f6ff';
    }

    function formatCard(input) {
       let val = input.value.replace(/\D/g, '').slice(0, 16);
       input.value = val.replace(/(.{4})/g, '$1  ').trim();
    }
</script>

{% endblock %}
```

3.5. TravelBot Assistant Snapshot and Coding

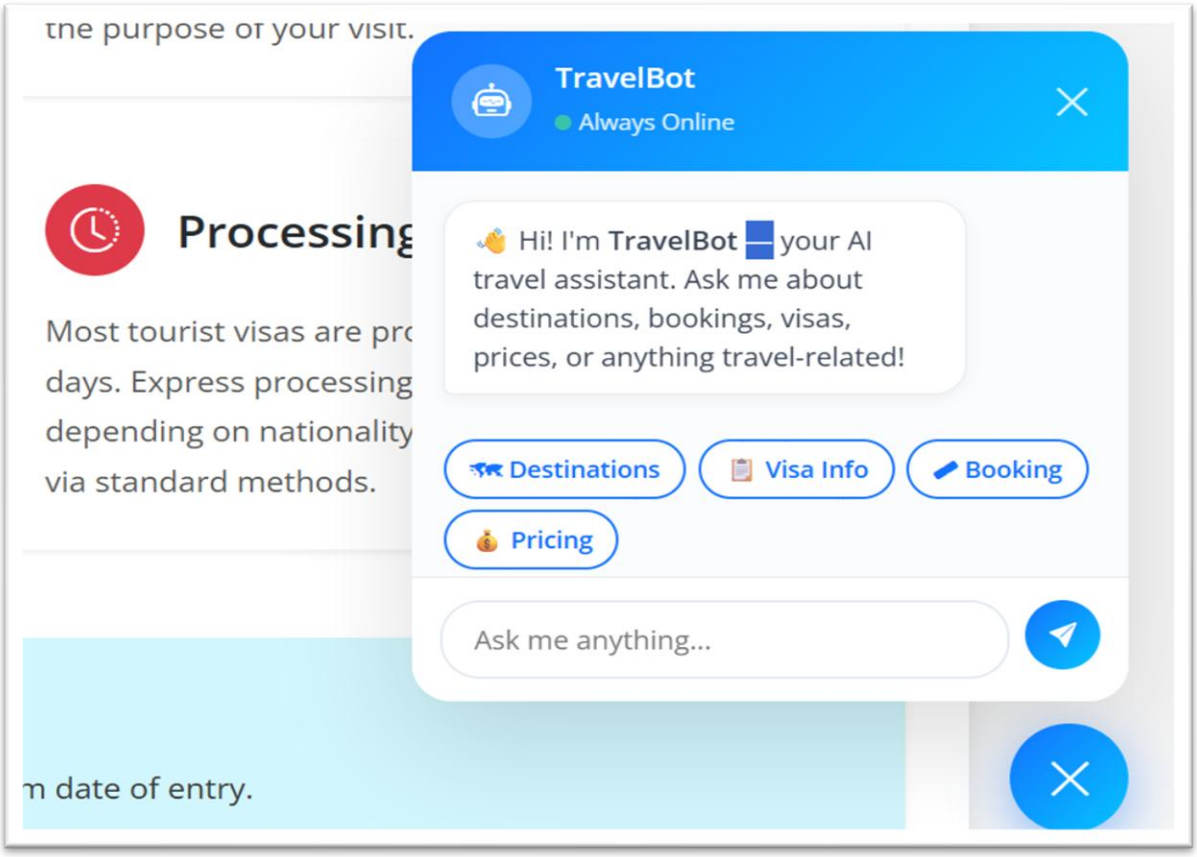

Fig 3.13. Travel Bot Assistant

```
<div class="ai-hero fade-in">
  <div class="container">
    <h1 class="display-4 fw-bold"><i class="bi bi-robot"></i> AI Trip Planner</h1>
    <p class="lead">Let our smart algorithm build your perfect day-by-day itinerary in seconds.</p>
  </div>
</div>

<div class="container fade-in">
  <div class="row justify-content-center">
    <div class="col-lg-8">
      <div class="planner-card">
        <form id="aiPlannerForm">
          <div class="row g-3">
            <div class="col-md-12">
              <label class="form-label fw-bold">Where do you want to go?</label>
              <input type="text" class="form-control" id="destination" placeholder="E.g., Paris,
Kyoto, or 'Surprise Me!'" required>
            </div>
            <div class="col-md-6">
              <label class="form-label fw-bold">Duration (Days)</label>
              <input type="number" class="form-control" id="days" min="1" max="14"
value="3" required>
```

```
                </div>
                <div class="col-md-6">
                  <label class="form-label fw-bold">Travel Vibe</label>
                  <select class="form-select" id="vibe">
                    <option value="Adventure">Adventure & Outdoors</option>
                    <option value="Relaxing">Relaxing & Spa</option>
                    <option value="Cultural">Cultural Immersion</option>
                    <option value="Foodie">Foodie Tour</option>
                    <option value="Balanced" selected>A bit of everything</option>
                  </select>
                </div>
              </div>
              <div class="text-center mt-4">
                <button type="submit" class="btn btn-primary btn-generate text-white w-100 fs-5">
                  <i class="bi bi-magic"></i> Generate My Itinerary
                </button>
              </div>
            </form>
          </div>
        </div>
      </div>

      <!-- Loading State -->
      <div class="ai-loader" id="aiLoader">
        <div class="spinner-grow text-primary" role="status">
          <span class="visually-hidden">Loading...</span>
        </div>
        <h4 class="mt-3 text-muted">Thinking contextually... mapping routes...</h4>
      </div>

      <!-- Result State -->
      <div id="itinerary-result" class="mb-5">
        <div id="res-hero" class="rounded-4 mb-4" style="height: 300px; background: center/cover no-
repeat; position: relative;">
          <div style="position: absolute; inset: 0; background: linear-gradient(to top, rgba(0,0,0,0.8),
transparent); border-radius: 20px;"></div>
          <div style="position: absolute; bottom: 30px; left: 30px; color: white;">
            <h2 class="fw-bold mb-0">Your <span id="res-vibe"></span> Trip to <span id="res-
dest"></span></h2>
            <p class="mb-0 opacity-75"><i class="bi bi-calendar-event me-2"></i>Curated by
TravelVista AI</p>
          </div>
          <div style="position: absolute; top: 20px; right: 20px;">
            <button class="btn btn-light rounded-pill px-3 shadow-sm" onclick="window.print()"><i
class="bi bi-printer me-2"></i>Print Plan</button>
          </div>
        </div>

        <div id="itinerary-days">
          <!-- Populated by JS -->
        </div>
      </div>
    </div>
```

```
<script>
    const destinationImages = {
        'paris': 'https://images.unsplash.com/photo-1502602898657-3e91760cbb34?q=80&w=1200',
        'tokyo': 'https://images.unsplash.com/photo-1540959733332-e94e270b4d82?q=80&w=1200',
        'london': 'https://images.unsplash.com/photo-1513635269975-59663e0ac1ad?q=80&w=1200',
        'dubai': 'https://images.unsplash.com/photo-1512453979798-5ea266f8880c?q=80&w=1200',
        'bali': 'https://images.unsplash.com/photo-1537996194471-e657df975ab4?q=80&w=1200',
        'mumbai': 'https://images.unsplash.com/photo-1566549097737-ed0df1d6962e?q=80&w=1200',
        'delhi': 'https://images.unsplash.com/photo-1587474260584-136574528ed5?q=80&w=1200',
        'goa': 'https://images.unsplash.com/photo-1512343879784-a960bf40e7f2?q=80&w=1200',
        'manali': 'https://images.unsplash.com/photo-1626621341517-bbf3d9990a23?q=80&w=1200',
        'kyoto': 'https://images.unsplash.com/photo-1493976040374-85c8e12f0c0e?q=80&w=1200',
        'default': 'https://images.unsplash.com/photo-1469854523086-cc02fe5d8800?q=80&w=1200'
    };

    document.getElementById('aiPlannerForm').addEventListener('submit', function(e) {
        e.preventDefault();

        const dest = document.getElementById('destination').value;
        const days = parseInt(document.getElementById('days').value);
        const vibe = document.getElementById('vibe').value;

        // Hide form, show loader
        document.querySelector('.planner-card').style.display = 'none';
        document.getElementById('aiLoader').style.display = 'block';
        document.getElementById('itinerary-result').style.display = 'none';

        // Simulate API Call
        setTimeout(() => {
            document.getElementById('aiLoader').style.display = 'none';
            document.getElementById('res-dest').innerText = dest;
            document.getElementById('res-vibe').innerText = vibe;

            // Set Hero Image
            const lowerDest = dest.toLowerCase();
            let bgImg = destinationImages.default;
            for(let key in destinationImages) {
                if(lowerDest.includes(key)) {
                    bgImg = destinationImages[key];
                    break;
                }
            }
            document.getElementById('res-hero').style.backgroundImage = `url('${bgImg}')`;

            generateMockItinerary(days, vibe);

            document.getElementById('itinerary-result').style.display = 'block';
        }, 2200);
    });

    function generateMockItinerary(days, vibe) {
        const container = document.getElementById('itinerary-days');
        container.innerHTML = '';
```

```
    const activities = {
        'Adventure': [
            { time: '08:00 AM', act: 'Sunrise Expedition & Exploration', icon: 'bi-brightness-alt-high' },
            { time: '11:00 AM', act: 'High-Altitude Aerial Activity', icon: 'bi-alt' },
            { time: '02:00 PM', act: 'Water-based Adventure Session', icon: 'bi-water' },
            { time: '05:00 PM', act: 'Sunset Trail & Photography', icon: 'bi-camera' },
            { time: '08:00 PM', act: 'Night-time Basecamp Social', icon: 'bi-stars' }
        ],
        'Relaxing': [
            { time: '09:00 AM', act: 'Mindfulness & Morning Serenity', icon: 'bi-heart-pulse' },
            { time: '11:00 AM', act: 'Artisanal Brunch Experience', icon: 'bi-cup-hot' },
            { time: '02:00 PM', act: 'Holistic Wellness & Spa Therapy', icon: 'bi-flower1' },
            { time: '05:00 PM', act: 'Coastal View Leisure Stroll', icon: 'bi-sunset' },
            { time: '08:00 PM', act: 'Premium Fine Dining Experience', icon: 'bi-moon-stars' }
        ],
        'Cultural': [
            { time: '09:00 AM', act: 'Heritage Corridor Guided Walk', icon: 'bi-bank' },
            { time: '12:00 PM', act: 'Local Artisanship Workshop', icon: 'bi-palette' },
            { time: '03:00 PM', act: 'Historical Landmark Navigation', icon: 'bi-building' },
            { time: '06:00 PM', act: 'Traditional Arts Performance', icon: 'bi-ticket-perforated' },
            { time: '08:30 PM', act: 'Curated Gastronomic Heritage Tour', icon: 'bi-egg-fried' }
        ],
        'Foodie': [
            { time: '08:00 AM', act: 'Iconic Local Market Breakfast', icon: 'bi-shop' },
            { time: '11:00 AM', act: 'Specialty Beverage Crafting Session', icon: 'bi-cup-hot' },
            { time: '01:00 PM', act: 'Signature Chef-led Tasting Lunch', icon: 'bi-egg-fried' },
            { time: '04:00 PM', act: 'Sweet Delights Discovery Route', icon: 'bi-box-seam' },
            { time: '08:00 PM', act: 'Premier Multiple-course Dinner', icon: 'bi-star-fill' }
        ],
        'Balanced': [
            { time: '09:00 AM', act: 'Primary City Landmark Exploration', icon: 'bi-camera' },
            { time: '12:00 PM', act: 'Authentic Local Sidewalk Cafe Experience', icon: 'bi-cup-hot' },
            { time: '02:30 PM', act: 'Curated Boutique & Craft Discovery', icon: 'bi-bag' },
            { time: '05:30 PM', act: 'Panoramic Vista Point Visit', icon: 'bi-camera-video' },
            { time: '08:00 PM', act: 'Social Hub Evening & Global Cuisine', icon: 'bi-stars' }
        ]
    };

    for(let i=1; i<=days; i++) {
        let html = `
            <div class="day-card animate-fade-in" style="animation-delay: ${i*0.1}s">
                <h4 class="day-header"><i class="bi bi-geo-fill me-2"></i> Day ${i}</h4>
        `;

        activities[vibe].forEach(event => {
            html += `
                <div class="activity">
                    <div class="activity-icon"><i class="bi ${event.icon}"></i></div>
                    <div>
                        <p class="mb-0 fw-bold text-dark">${event.time}</p>
                        <p class="text-muted mb-0">${event.act}</p>
                    </div>
                </div>
            `;
```

```
        });

        html += `</div>`;
        container.innerHTML += html;
      }

      container.innerHTML += `
        <div class="text-center mt-4">
          <button class="btn btn-primary rounded-pill px-4 shadow-sm" onclick="resetPlanner()">
            <i class="bi bi-arrow-left me-2"></i>Plan Another Journey
          </button>
        </div>
      `;
    }

    function resetPlanner() {
      document.getElementById('itinerary-result').style.display = 'none';
      document.querySelector('.planner-card').style.display = 'block';
      document.getElementById('destination').value = '';
    }
</script>

{% endblock %}
```

3.6. User Profile & Setting Snapshot and Coding

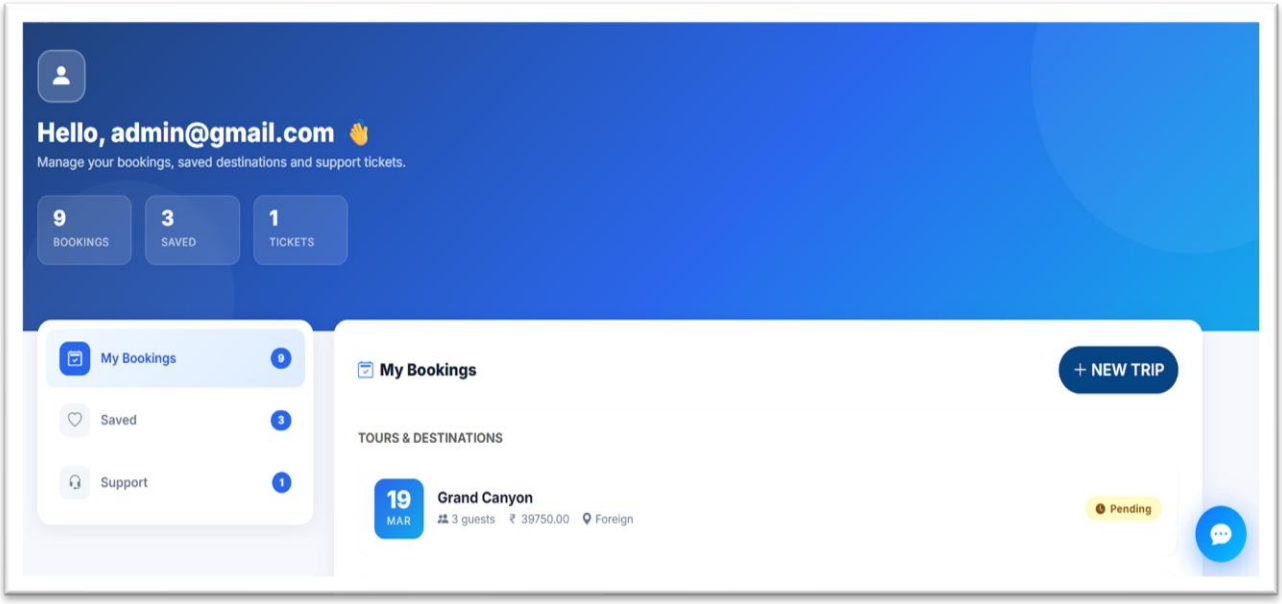

Fig 3.14. User Profile

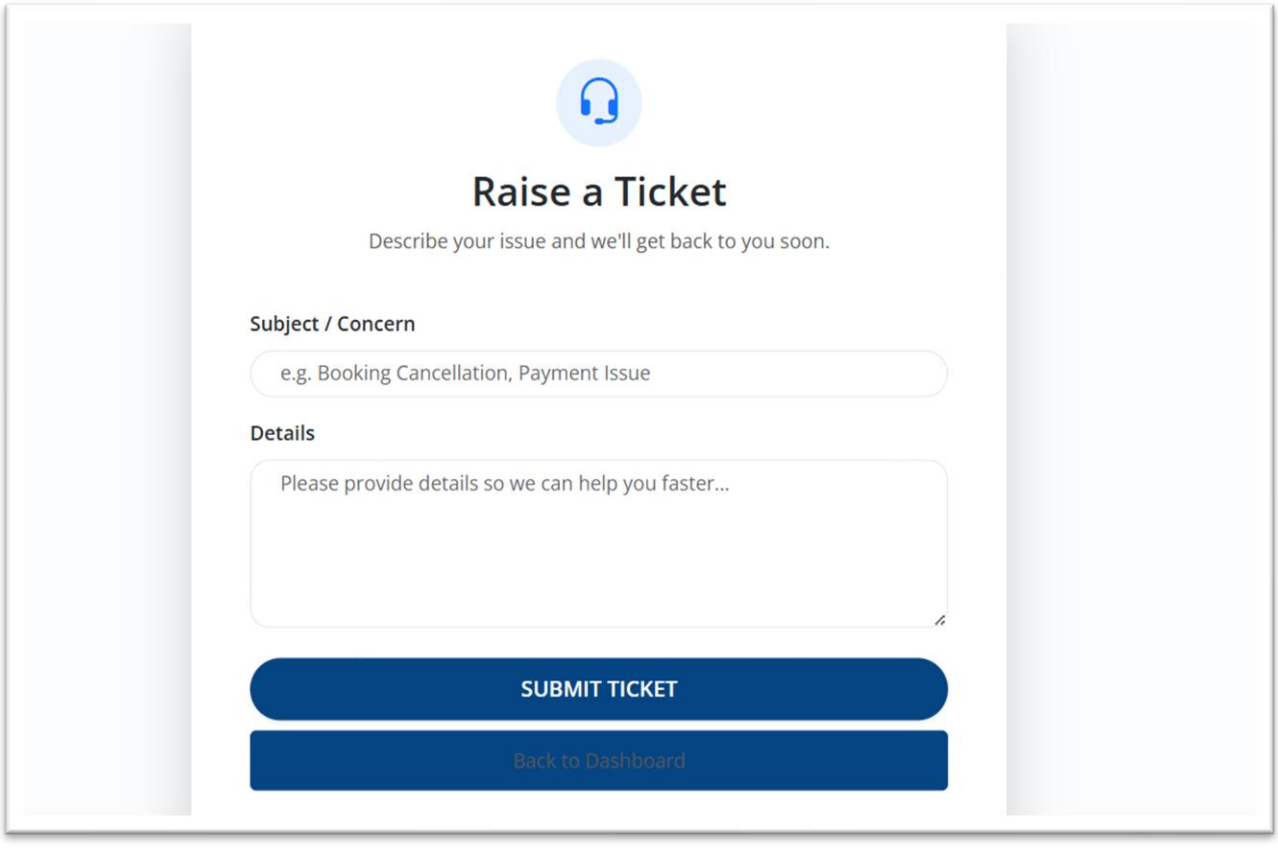

Fig 3.15. Raise issue page

```
{% extends 'main/base.html' %}
{% load static %}

{% block title %}Raise Support Ticket | TravelVista{% endblock %}

{% block content %}
<section class="py-5 bg-light">
  <div class="container">
    <div class="row justify-content-center">
      <div class="col-md-6">
        <div class="card border-0 shadow-lg rounded-4 p-4 p-md-5">
          <div class="text-center mb-4">
            <div class="bg-primary bg-opacity-10 text-primary rounded-circle d-inline-flex align-
  items-center justify-content-center mb-3"
              style="width: 70px; height: 70px;">
              <i class="bi bi-headset fs-1"></i>
            </div>
            <h2 class="fw-bold">Raise a Ticket</h2>
            <p class="text-muted">Describe your issue and we'll get back to you soon.</p>
          </div>

          <form method="POST">
            {% csrf_token %}
            <div class="mb-3">
              <label class="form-label fw-bold">Subject / Concern</label>
              <input type="text" name="subject" class="form-control rounded-pill px-4"
                placeholder="e.g. Booking Cancellation, Payment Issue" required>
            </div>
            <div class="mb-4">
              <label class="form-label fw-bold">Details</label>
                <textarea name="message" class="form-control rounded-4 px-4" rows="5"
                    placeholder="Please provide details so we can help you faster..."
        required></textarea>
              </div>
            <div class="d-grid gap-2">
                  <button type="submit" class="btn btn-primary btn-lg rounded-pill fw-bold
        shadow-sm">Submit
                Ticket</button>
              <a href="{% url 'user_dashboard' %}"
                class="btn btn-link text-decoration-none text-muted">Back to Dashboard</a>
            </div>
          </form>
        </div>
      </div>
    </div>
  </div>
</section>
{% endblock %}
```

```
from django.urls import path
from . import views

urlpatterns = [
    # Homepage
    path('', views.home, name='home'),

    # Contact page with form submission
    path('contact/', views.contact, name='contact'),
    path('contact/success/', views.contact_success, name='contact_success'),

    # Visa policy page
    path('visa-policy/', views.visa_policy, name='visa_policy'),

    # How to Reach section
    path('how-to-reach/', views.how_to_reach, name='how_to_reach'),
    path('how-to-reach/train/', views.by_train, name='by_train'),
    path('how-to-reach/bus/', views.by_bus, name='by_bus'),
    path('how-to-reach/flight/', views.by_flight, name='by_flight'),

    # Destination section
    path('destination/', views.destination_list, name='destination'),

    path('destination/regional/', views.regional, name='regional'),
    path('destination/foreign/', views.foreign, name='foreign'),
    path('destinations/', views.destination_list, name='destination_list'),

    # Best time to visit page
    path('best-time/', views.best_time, name='best_time'),

    # Auth routes
    path('register/', views.register_user, name='register'),
    path('login/', views.login_user, name='login_user'),
    path('logout/', views.logout_user, name='logout_user'),

    # Booking & Payment
    path('book/<int:dest_id>/', views.book_destination, name='book_destination'),
    path('payment/<int:booking_id>/', views.process_payment, name='process_payment'),

    # Unique Features
    path('mood/', views.mood_filter, name='mood_filter'),
    path('ai-planner/', views.ai_planner, name='ai_planner'),
    path('buddy-matcher/', views.buddy_matcher, name='buddy_matcher'),

    # User Dashboard & Ticketing
    path('dashboard/', views.user_dashboard, name='user_dashboard'),
    path('dashboard/cancel-booking/<int:booking_id>/', views.cancel_booking,
name='cancel_booking'),
    path('dashboard/save-destination/<int:dest_id>/', views.save_destination,
name='save_destination'),
```

```
    path('dashboard/remove-saved/<int:saved_id>/', views.remove_saved, name='remove_saved'),
    path('dashboard/tickets/create/', views.create_ticket, name='create_ticket'),

    # Hotel Booking System
    path('hotels/', views.hotel_list, name='hotel_list'),
    path('hotels/book/<int:hotel_id>/', views.book_hotel, name='book_hotel'),
    path('hotels/payment/<int:booking_id>/', views.process_hotel_payment,
name='process_hotel_payment'),
]



div class="db-page">
    <!-- Header -->
    <div class="db-header">
        <div class="container db-header-inner">
            <div class="db-avatar"><i class="bi bi-person-fill"></i></div>
            <h1>Hello, {{ user.username }} 👋</h1>
            <p>Manage your bookings, saved destinations and support tickets.</p>

            <div class="db-stats">
                <div class="db-stat">
                    <strong>{{ bookings.count|add:hotel_bookings.count }}</strong>
                    <span>Bookings</span>
                </div>
                <div class="db-stat">
                    <strong>{{ saved_destinations.count }}</strong>
                    <span>Saved</span>
                </div>
                <div class="db-stat">
                    <strong>{{ tickets.count }}</strong>
                    <span>Tickets</span>
                </div>
            </div>
        </div>
    </div>

    <!-- Body -->
    <div class="db-body">
        <div class="container">
            <div class="row g-4">

                <!-- Sidebar -->
                <div class="col-lg-3">
                    <div class="db-sidebar">
                        <button class="db-nav-btn active" onclick="switchSection('bookings', this)">
                            <div class="db-nav-icon"><i class="bi bi-calendar2-check"></i></div>
                            My Bookings
                            {% with total_count=bookings.count|add:hotel_bookings.count %}
                            {% if total_count > 0 %}<span class="db-nav-badge">{{ total_count }}</span>{%
endif %}
                            {% endwith %}
                        </button>
                        <button class="db-nav-btn" onclick="switchSection('saved', this)">
```

```
                    <div class="db-nav-icon"><i class="bi bi-heart"></i></div>
                    Saved
                    {% if saved_destinations.count > 0 %}<span class="db-nav-badge">{{
saved_destinations.count
                      }}</span>{% endif %}
                  </button>
                  <button class="db-nav-btn" onclick="switchSection('tickets', this)">
                    <div class="db-nav-icon"><i class="bi bi-headset"></i></div>
                    Support
                    {% if tickets.count > 0 %}<span class="db-nav-badge">{{ tickets.count
}}</span>{% endif %}
                  </button>
                </div>
              </div>

              <!-- Main -->
              <div class="col-lg-9">

                <!-- BOOKINGS -->
                <div class="db-section active" id="sec-bookings">
                  <div class="db-panel">
                    <div class="db-panel-header">
                      <h4><i class="bi bi-calendar2-check me-2 text-primary"></i>My Bookings</h4>
                      <a href="{% url 'destination_list' %}" class="btn btn-primary btn-sm rounded-pill
                        px-3">
                        <i class="bi bi-plus-lg me-1"></i>New Trip
                      </a>
                    </div>

                    {% if bookings or hotel_bookings %}
                    {% if bookings %}
                    <h6 class="fw-bold mb-3 text-muted small text-uppercase">Tours &
                        Destinations</h6>
                    <div class="d-flex flex-column gap-3 mb-4">
                      {% for booking in bookings %}
                      <div class="bk-card">
                        <div class="bk-date-box">
                          <div class="day">{{ booking.booking_date|date:"d" }}</div>
                          <div class="mon">{{ booking.booking_date|date:"M" }}</div>
                        </div>
                        <div class="bk-info">
                          <h5>{{ booking.destination.name }}</h5>
                          <div class="bk-meta">
                            <span><i class="bi bi-people-fill"></i> {{ booking.guests }} guest{% if
                              booking.guests > 1 %}s{% endif %}</span>
                            <span><i class="bi bi-currency-rupee"></i> {{ booking.total_price
                              }}</span>
                            <span><i class="bi bi-geo-alt-fill"></i> {{ booking.destination.region
                              }}</span>
                          </div>
                        </div>
                        <div class="bk-actions">
                          {% if booking.payment_status == 'Completed' %}
                          <span class="st-badge st-completed"><i class="bi bi-check-circle-fill"></i>
```

```
                        Completed</span>
                    {% elif booking.payment_status == 'Cancelled' %}
                    <span class="st-badge st-cancelled"><i class="bi bi-x-circle-fill"></i>
                        Cancelled</span>
                    {% else %}
                    <span class="st-badge st-pending"><i class="bi bi-clock-fill"></i>
                        Pending</span>
                    {% endif %}
                </div>
            </div>
            {% endfor %}
        </div>
    {% endif %}
    {% if hotel_bookings %}
        <h6 class="fw-bold mb-3 text-muted small text-uppercase">Hotel Stays</h6>
        <div class="d-flex flex-column gap-3">
            {% for booking in hotel_bookings %}
            <div class="bk-card">
                <div class="bk-date-box"
                    style="background: linear-gradient(135deg, #10b981, #059669);">
                    <div class="day">{{ booking.check_in|date:"d" }}</div>
                    <div class="mon">{{ booking.check_in|date:"M" }}</div>
                </div>
                <div class="bk-info">
                    <h5>{{ booking.hotel.name }}</h5>
                    <div class="bk-meta">
                        <span><i class="bi bi-geo-alt"></i> {{ booking.hotel.destination.name
                            }}</span>
                        <span><i class="bi bi-people"></i> {{ booking.guests }} guests</span>
                        <span><i class="bi bi-calendar-range"></i> {{ booking.check_in }} to {{
                            booking.check_out }}</span>
                    </div>
                </div>
                <div class="bk-actions">
                    {% if booking.payment_status == 'Completed' %}
                    <span class="st-badge st-completed"><i class="bi bi-check-circle-fill"></i>
                        Confirmed</span>
                    {% else %}
                    <span class="st-badge st-pending"><i class="bi bi-clock-fill"></i> {{
                        booking.payment_status }}</span>
                    {% endif %}
                    <div class="fs-6 fw-bold text-success">₹{{ booking.total_price }}</div>
                </div>
            </div>
            {% endfor %}
        </div>
        {% endif %}
        {% else %}
        <div class="db-empty">
            <div class="empty-icon"><i class="bi bi-calendar-x"></i></div>
            <h5>No Bookings Yet</h5>
            <p>Start exploring destinations and book your next adventure!</p>
    <a href="{% url 'destination_list' %}" class="btn btn-primary rounded-pill px-4">Browse
            Destinations</a>
```

```
                </div>
                {% endif %}
              </div>
            </div>

            <!-- SAVED DESTINATIONS -->
            <div class="db-section" id="sec-saved">
              <div class="db-panel">

 <div class="db-panel-header">
                  <h4><i class="bi bi-heart-fill me-2 text-danger"></i>Saved Destinations</h4>
                  <a href="{% url 'destination_list' %}"
                    class="btn btn-outline-primary btn-sm rounded-pill px-3">Explore More</a>
                </div>

                {% if saved_destinations %}
                <div class="row g-3">
                  {% for saved in saved_destinations %}
                  <div class="col-sm-6">
                    <div class="sv-card position-relative">
                      {% if saved.destination.image %}
                 <img src="{{ saved.destination.image.url }}" alt="{{ saved.destination.name }}">
                      {% else %}
                          <img src="https://images.unsplash.com/photo-1476514525535-
                 07fb3b4ae5f1?q=80&w=800&auto=format&fit=crop"
                        alt="{{ saved.destination.name }}">
                      {% endif %}
                      <a href="{% url 'remove_saved' saved.id %}" class="sv-remove"
                 title="Remove"><i
                          class="bi bi-x-lg"></i></a>
                      <div class="sv-card-body">
                        <h6>{{ saved.destination.name }}</h6>
                        <p class="text-muted small mb-2">{{
                          saved.destination.description|truncatewords:10 }}</p>
                        <div class="d-flex justify-content-between align-items-center">
                          <span class="fw-bold text-success small">₹{{ saved.destination.price }}
                            / person</span>
                          <a href="{% url 'book_destination' saved.destination.id %}"
                            class="btn btn-primary btn-sm rounded-pill px-3">Book</a>
                        </div>
                      </div>
                    </div>
                  </div>
                  {% endfor %}
                </div>
                {% else %}
                <div class="db-empty">
                  <div class="empty-icon"><i class="bi bi-heartbreak"></i></div>
                  <h5>Nothing Saved Yet</h5>
                  <p>Click the heart icon on any destination to save it here for later.</p>
                  <a href="{% url 'destination_list' %}"
                    class="btn btn-outline-primary rounded-pill px-4">Find Destinations</a>
                </div>
                {% endif %}
```

```
            </div>
        </div>

        <!-- SUPPORT TICKETS -->
        <div class="db-section" id="sec-tickets">
            <div class="db-panel">
                <div class="db-panel-header">
                    <h4><i class="bi bi-headset me-2 text-primary"></i>Support Tickets</h4>
                    <a href="{% url 'create_ticket' %}" class="btn btn-dark btn-sm rounded-pill px-3">

            <i class="bi bi-plus-lg me-1"></i>New Ticket
                    </a>
                </div>

                {% if tickets %}
                <div>
                    {% for ticket in tickets %}
                    <div class="tk-row">
                        <div class="tk-icon"><i class="bi bi-ticket-perforated-fill"></i></div>
                        <div class="tk-info">
                            <strong>{{ ticket.subject }}</strong>
                            <span>{{ ticket.message|truncatewords:10 }}</span>
                        </div>
                        <div class="d-flex flex-column align-items-end gap-1">
                            {% if ticket.status == 'Open' %}
                            <span class="st-badge st-open"><i class="bi bi-circle-fill"
                                    style="font-size:0.5rem;"></i> Open</span>
                            {% elif ticket.status == 'In Progress' %}
                            <span class="st-badge st-inprogress"><i class="bi bi-arrow-repeat"></i> In
                                Progress</span>
                            {% else %}
                        <span class="st-badge st-closed"><i class="bi bi-check2-all"></i> Closed</span>
                            {% endif %}
                            <span class="text-muted" style="font-size: 0.72rem;">{{
                                ticket.created_at|date:"d M Y" }}</span>
                        </div>
                    </div>
                    {% endfor %}
                </div>
                {% else %}
                <div class="db-empty">
                    <div class="empty-icon"><i class="bi bi-chat-dots"></i></div>
                    <h5>No Tickets Yet</h5>
                    <p>Need help with a booking or have a question? Raise a support ticket.</p>
                    <a href="{% url 'create_ticket' %}" class="btn btn-dark rounded-pill px-4">Create
                        Ticket</a>
                </div>
                {% endif %}
            </div>
        </div>

    </div>
</div>
```

```
    </div>
</div>
```

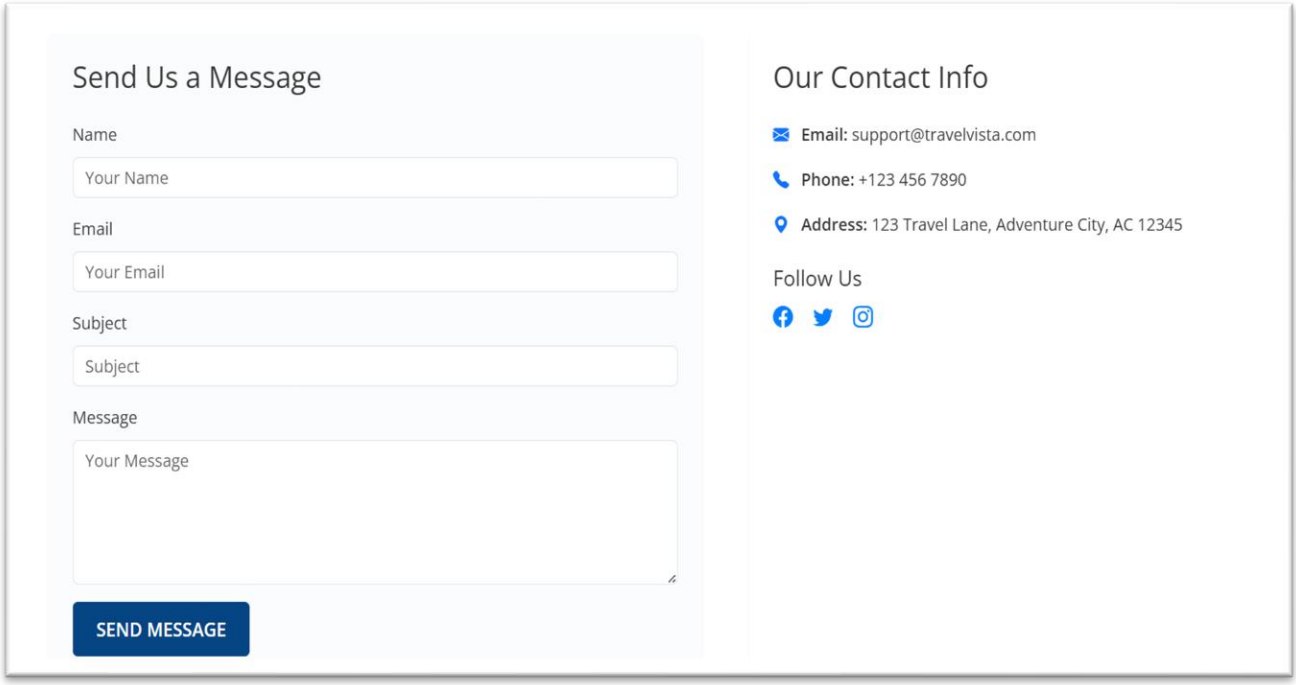

Fig 3.16. Contact Us

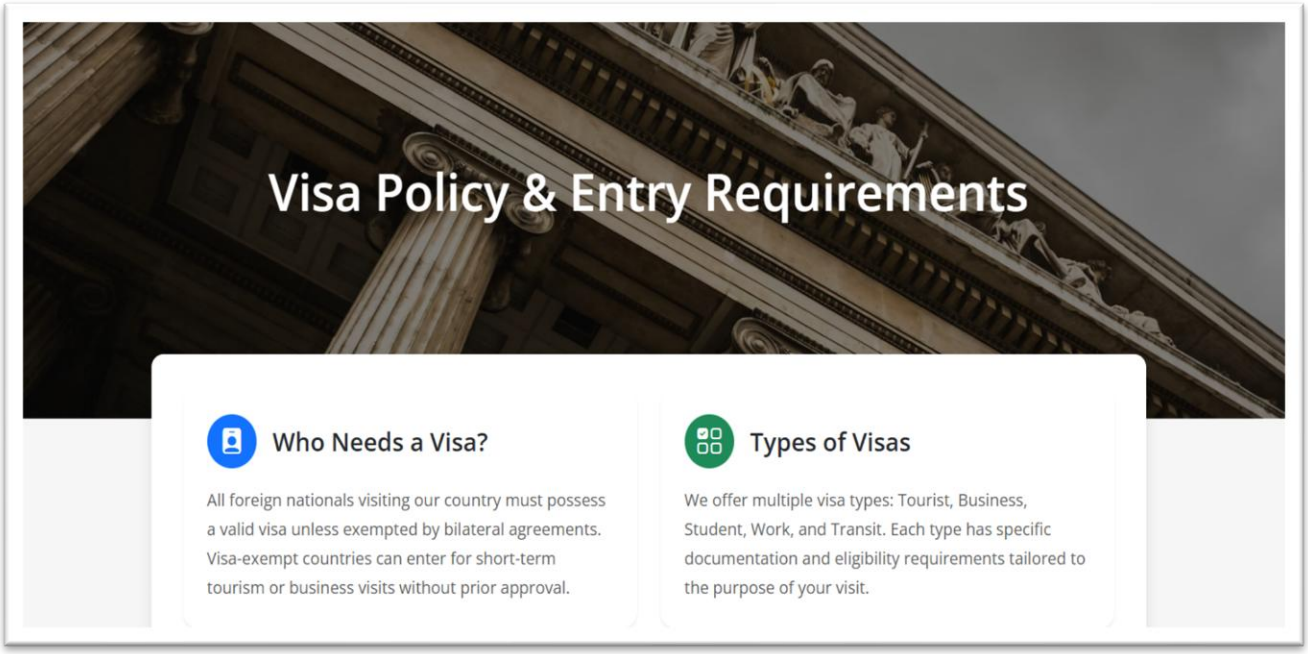

Fig 3.17. Visa & Policy

```
<section class="visa-content container fade-in mt-n5 position-relative z-3">

 <div class="row g-4 mb-5">
  <!-- Who Needs a Visa -->
  <div class="col-md-6">
   <div class="card h-100 border-0 shadow-sm visa-card">
    <div class="card-body p-4">
     <div class="d-flex align-items-center mb-3">
      <div class="icon-circle bg-primary text-white me-3"><i class="bi bi-person-badge-fill fs-
4"></i></div>
      <h3 class="card-title h4 mb-0 fw-bold">Who Needs a Visa?</h3>
     </div>
     <p class="card-text text-muted">All foreign nationals visiting our country must possess a valid
visa unless
      exempted by bilateral agreements. Visa-exempt countries can enter for short-term tourism or
business visits
      without prior approval.</p>
    </div>
   </div>
  </div>

  <!-- Types of Visas -->
  <div class="col-md-6">
   <div class="card h-100 border-0 shadow-sm visa-card">
    <div class="card-body p-4">
     <div class="d-flex align-items-center mb-3">
      <div class="icon-circle bg-success text-white me-3"><i class="bi bi-ui-checks-grid fs-
4"></i></div>
      <h3 class="card-title h4 mb-0 fw-bold">Types of Visas</h3>
     </div>
     <p class="card-text text-muted">We offer multiple visa types: Tourist, Business, Student,
Work, and Transit.
      Each type has specific documentation and eligibility requirements tailored to the purpose of
your visit.</p>
    </div>
   </div>
  </div>

  <!-- Application Process -->
  <div class="col-md-6">
   <div class="card h-100 border-0 shadow-sm visa-card">
    <div class="card-body p-4">
     <div class="d-flex align-items-center mb-3">
      <div class="icon-circle bg-warning text-dark me-3"><i class="bi bi-laptop fs-4"></i></div>
      <h3 class="card-title h4 mb-0 fw-bold">Online Application</h3>
     </div>
     <p class="card-text text-muted">Applications can be submitted online through the official visa
portal.
      Applicants must upload a passport-sized photo, scanned passport, travel itinerary, and proof of
      accommodation.</p>
    </div>
   </div>
  </div>
```

```
  <!-- Processing & Fees -->
  <div class="col-md-6">
    <div class="card h-100 border-0 shadow-sm visa-card">
      <div class="card-body p-4">
        <div class="d-flex align-items-center mb-3">
          <div class="icon-circle bg-danger text-white me-3"><i class="bi bi-clock-history fs-
4"></i></div>
          <h3 class="card-title h4 mb-0 fw-bold">Processing & Fees</h3>
        </div>
        <p class="card-text text-muted">Most tourist visas are processed within 3–7 working days.
Express processing
          is available. Fees vary depending on nationality. Payment is accepted online via standard
methods.</p>
      </div>
    </div>
  </div>
 </div>

 <!-- Important Notes Alert Box -->
 <div class="alert alert-info border-0 shadow-sm p-4 mb-5 rounded-4 d-flex align-items-start fade-
in">
  <i class="bi bi-info-circle-fill fs-1 text-info me-4 mt-1"></i>
  <div>
    <h4 class="alert-heading fw-bold mb-3 text-dark">Important Notes Before Travel</h4>
    <ul class="list-unstyled mb-0 text-dark">
      <li class="mb-2"><i class="bi bi-check2-circle text-primary me-2"></i> Passport must be valid
for at least 6
        months from date of entry.</li>
      <li class="mb-2"><i class="bi bi-check2-circle text-primary me-2"></i> A Visa does not
guarantee entry.
        Immigration officers have the final say.</li>
      <li><i class="bi bi-check2-circle text-primary me-2"></i> Overstaying is a serious offense and
may lead to fines
        or bans.</li>
    </ul>
  </div>
 </div>

 <div class="row g-4 mb-5">
  <div class="col-12 text-center mt-5 mb-4">
    <h2 class="fw-bold">Destination Visa Guides</h2>
    <p class="text-muted">Quick visa facts for our top international and regional locations.</p>
  </div>
  {% for dest in destinations %}
  <div class="col-md-4">
    <div class="card h-100 border-0 shadow-sm visa-card" style="background: #fdfdfd;">
      <div class="card-body p-4">
        <h5 class="fw-bold mb-3"><i class="bi bi-geo-alt-fill text-primary me-2"></i>{{ dest.name
}}</h5>
        <p class="card-text text-muted small" style="line-height: 1.6;">{{ dest.visa_info }}</p>
        <div class="mt-3 pt-3 border-top">
          <span class="badge bg-light text-dark border">Region: {{ dest.region }}</span>
        </div>
```

```
  </div>
 </div>
```

CHAPTER 4

TESTING AND EVALUATION

Testing is an important phase in the software development lifecycle that ensures the TravelVista Tourist Management System operates correctly, securely, and efficiently according to user requirements. Since TravelVista is a web-based tourism platform that handles user authentication, destination browsing, tour package management, and booking processes, comprehensive testing was necessary to validate system performance, reliability, and security.

Various testing techniques were applied during the development process to identify errors, verify system functionality, and ensure smooth interaction between tourists and administrators.

4.1 Types of Testing Performed

4.1.1 Unit Testing

Unit testing focuses on verifying individual components of the TravelVista system independently. Each module was tested separately to ensure that it performs the intended functionality.

Backend modules developed using Python and Django were tested individually, including:

- User authentication functions
- Destination management operations
- Tour package handling
- Booking management processes
- Database interaction functions

Frontend components developed using **HTML, CSS, and Bootstrap** such as login forms, registration pages, destination views, and booking forms were also tested individually.

Unit testing helped detect errors at an early stage and ensured that each module functioned correctly before integrating them into the complete system.

4.1.2 Integration Testing

Integration testing was conducted to ensure proper interaction between different modules of the TravelVista Tourist Management System.

For example, when a user searches for tourist destinations through the interface, the system processes the request, retrieves destination information from the database, and displays the results correctly.

Similarly, booking requests, user authentication processes, and destination management

operations were tested to ensure seamless communication between the frontend interface, backend application, and database

4.1.3 Functional Testing

Functional testing was performed to verify that all functional requirements of the TravelVista system were satisfied.

Key functionalities tested include:

- User registration and login
- Searching tourist destinations
- Viewing travel packages
- Booking tour packages
- Administrator management of destinations and bookings

Different user scenarios were simulated to ensure that the system produced the expected results under various conditions. Functional testing was mainly conducted manually through the web interface to simulate real user interactions.

4.1.4 System Testing

System testing was conducted to evaluate the TravelVista Tourist Management System as a complete integrated application. All modules were tested together to ensure that the system performs correctly when used in real-world scenarios.

End-to-end workflows such as user registration, destination browsing, tour package viewing, booking requests, and booking confirmation were tested to verify smooth operation. Various edge cases such as invalid input data, duplicate bookings, incorrect login credentials, and session expiration were also tested.

4.1.5 Security Testing

The authentication system implemented using Django’s built-in authentication framework was tested to verify secure login and session management. Password encryption and secure login mechanisms were validated to ensure that user credentials remain protected..

Input validation was implemented and tested to prevent invalid data submission and potential security vulnerabilities. Access control mechanisms were also tested to ensure that only authorized users could access specific system functionalities.

4.2 Evaluation Criteria

4.2.1 Accuracy

Accuracy was evaluated by verifying that property details, booking information, and payment status were correctly processed and stored in the database. Test cases confirmed that system outputs matched user inputs without data loss or inconsistencies.

4.2.2 Efficiency

Efficiency was measured based on system response time during user actions such as login, property search, and booking confirmation. Optimized backend APIs and efficient database queries ensured fast response times, even when multiple users accessed the system simultaneously.

4.2.3 Usability

Usability evaluation focused on how easily users could interact with the TravelVista Tourist Management System. The user interface was evaluated for simplicity, responsiveness, and ease of navigation. The system provides a clear layout, well-organized menus, and intuitive controls that allow users to easily search destinations, view tour packages, and make bookings. The responsive design ensures that the system works efficiently on desktops, tablets, and mobile devices, improving the overall user experience.

4.2.4 Reliability

Reliability testing ensured that the TravelVista system performed consistently during normal operation and under different usage conditions. The system was tested for stability during repeated destination searches, booking requests, and temporary network interruptions. The application demonstrated stable performance with proper error handling and reliable database operations.

4.2.5 Security

Security evaluation verified the effectiveness of authentication, authorization, and data protection mechanisms implemented in the system. Unauthorized access attempts were restricted, user credentials were securely stored, and session management policies were enforced. These security measures ensure that sensitive user information and booking data remain protected.

4.2.6 Scalability

Although the system is initially designed for a moderate number of users, TravelVista was evaluated for scalability. The modular architecture of the system allows it to support additional users, destinations, and booking records without major structural changes. This makes the system suitable for future expansion and increased usage.

4.2.7 Maintainability

Maintainability was evaluated based on code structure, documentation quality, and technology selection. The use of Python, Django, HTML, CSS, and Bootstrap provides a clean and modular development structure. This makes it easier for developers to implement future updates, fix bugs, and add new features without affecting existing system functionality.

CHAPTER 5

TASK ANALYSIS AND SCHEDULE OF ACTIVITIES

5.1 Task Decomposition

Task decomposition for the TravelVista Tourist Management System project was performed by dividing the system into logical modules related to tourists and administrators. This approach helped in organizing development activities, monitoring project progress, and ensuring efficient use of time and resources.

5.1.1 Requirement Analysis

- Identify user needs for a room rental and accommodation platform
- Define functional requirements such as user registration, property listing, room search, booking, and payment
- Define non-functional requirements such as security, scalability, and performance
- Prepare user flow diagrams and interaction scenarios

5.1.2 System Design

- Design UI layouts and wireframes for login, destination browsing, and booking pages
- Define system architecture and application workflow
- Design database schema for users, destinations, tour packages, and booking records

5.1.3 Frontend Development

- Develop responsive user interfaces using HTML, CSS, and Bootstrap
- Implement pages for user registration, login, destination search, and booking
- Design database schema for users, properties, bookings, and payments

5.1.4 Backend Development

- Develop backend functionality using Python and Django framework
- Implement business logic for destination management, package display, and booking processes
- Integrate authentication and role-based access control

5.1.5 Database Configuration

- Configure SQLite database for storing application data
- Manage collections for users, properties, bookings, and transactions
- Ensure efficient read and write operations

5.1.6 Security & Session Management

- Implement secure user authentication using Django's authentication system authentication
- Protect sensitive user information and booking data
- Verify access permissions for tourists and administrators

5.1.7 Testing

- Perform unit, integration, and system testing
- Validate functionality such as booking flow and payment processing
- Handle edge cases and error scenarios

5.1.8 Deployment

- Deploy frontend and backend services on cloud platforms PythonAnywhere or Render.
- Configure environment variables and database connections
- Monitor system performance after deployment

5.1.9 Documentation

- Prepare complete project documentation and technical reports
- Create user guides and developer documentation for future maintenance
- Create user guide and developer references

5.2 Project Schedule

The project schedule defines the sequence and duration of development activities to ensure timely completion of the TravelVista Tourist Management system. Each phase was planned with realistic time estimates and clear dependencies, allowing smooth progress from requirement analysis to deployment.

Fig 5.1. Project Schedule

5.3 Task Specification

Task specification clearly defines each major development activity in terms of objectives, duration, dependencies, and expected outcomes. This helps in efficient time management, resource allocation, and progress tracking throughout the project lifecycle.

5.3.1 Requirement Analysis

- **Goal:** Identify user requirements for the TravelVista platform and define functional and non-functional requirements
- **Duration:** Week 1 – Week 2
- **Dependencies:** None

5.3.2 System Design

- **Goal:** Design overall system architecture, data flow, and database schema
- **Duration:** Week 2 – Week 3
- **Dependencies:** Requirement Analysis

5.3.3 Frontend Development

- **Goal:** Develop responsive user interfaces for room search, property listing, and booking features
- **Duration:** Week 3 – Week 6
- **Dependencies:** System Design

5.3.4 Backend Development

- **Goal:** Implement APIs, authentication, booking logic, and business rules
- **Duration:** Week 4 – Week 7
- **Dependencies:** System Design

5.3.5 Database Development

- **Goal:** Design and implement database collections for users, properties, and bookings
- **Duration:** Week 5 – Week 7
- **Dependencies:** Backend Development

5.3.6 Module Integration

- **Goal:** Integrate frontend, backend, and database modules into a unified system
- **Duration:** Week 6 – Week 8
- **Dependencies:** Frontend and Backend Development

5.3.7 Testing

- **Goal:** Verify system functionality, performance, and security through testing
- **Duration:** Week 8 – Week 9
- **Dependencies:** Module Integration

5.3.8 Deployment

- **Goal:** Deploy the TravelVista Tourist Management system on a cloud platform and make it available for users
- **Duration:** Week 9 – Week 10
- **Dependencies:** Testing

5.3.9 Documentation

- **Goal:** Prepare complete project documentation, user manuals, and technical reports
- **Duration:** Week 10
- **Dependencies:** Deployment

CHAPTER 6

PROJECT MANAGEMENT

Project management played an important role in the successful development of the TravelVista Tourist Management System. The project followed a structured development lifecycle including requirement analysis, system design, development, testing, and deployment.

A flexible Agile-inspired approach was adopted to allow continuous improvements during development. Each phase of the project was treated as a milestone, which helped identify issues early and ensured smooth progress throughout the development process. This approach improved project efficiency, system quality, and risk management.

6.1 Project Planning

Project planning formed the foundation of the TravelVista Tourist Management System development process. At the beginning of the project, clear objectives were defined, including:

- Developing a user-friendly tourism management platform
- Enabling secure user authentication and role-based access
- Allowing administrators to manage tourist destinations and tour packages
- Providing efficient room search and booking functionality
- Ensuring secure data storage and system reliability

A detailed project roadmap was prepared outlining major tasks, dependencies, and estimated timelines. This roadmap helped distribute work efficiently across different development phases and ensured that critical components such as authentication, booking management, and database operations were prioritized.

Task Breakdown and Work Allocation

To manage system complexity, the TravelVista project was divided into smaller manageable tasks using task decomposition. Each task was assigned specific objectives and expected outcomes. The major task categories included:

- Frontend development (user interface for destination search and booking)
- Backend development (application logic using Django)
- Database management (user, Destination, and booking data)
- Authentication and authorization implementation
- Booking and payment management
- Testing and debugging
- Deployment and documentation

This structured breakdown improved progress tracking and ensured that no critical module was overlooked. Tasks were executed sequentially where dependencies existed and in parallel wherever possible to optimize development time.

Monitoring and Progress Tracking

Continuous monitoring was essential to ensure that the project remained on schedule. Simple project tracking methods such as task lists, progress charts, and milestone reviews were used to monitor completed and pending tasks.

Regular self-evaluation was conducted after completing each module to verify that the implementation met functional and quality requirements. Monitoring also ensured proper handling of dependencies, such as confirming backend readiness before frontend integration and validating database operations before deployment.

Quality Management

Quality management ensured that the TravelVista system met the expected standards of performance, reliability, and usability. Quality was maintained through:

- Regular testing during each development phase
- Code reviews to improve readability and maintainability
- Validation of booking workflows and data accuracy
- UI testing to ensure smooth and intuitive user interaction

By integrating testing activities throughout development rather than postponing them to the end, issues were identified early and resolved efficiently.

Risk Management

Risk management was an important part of the TravelVista project planning process. Potential risks were identified early and preventive measures were planned to reduce their impact.

- Misinterpretation of user requirements
- Authentication or authorization failures
- Booking conflicts or data inconsistency
- Security vulnerabilities
- Deployment or configuration issues

These risks were mitigated through regular requirement validation, modular system design, secure coding practices, and frequent testing. This proactive approach ensured smoother execution and reduced uncertainty during development.

Communication and Coordination

Effective communication and coordination were maintained throughout the project lifecycle. Design decisions, implementation strategies, and encountered challenges were documented clearly to ensure consistency across modules.

Coordination between frontend, backend, and database components was achieved through well-defined APIs and structured data models. This reduced integration errors and improved overall development efficiency.

Change Management

As development progressed, certain changes and enhancements were identified based on testing results and usability feedback. Instead of resisting changes, the project management strategy allowed controlled modifications.

Each change was evaluated for its impact on system stability and project timelines. Critical improvements were implemented immediately, while non-essential enhancements were postponed to maintain deadlines. This ensured a balance between flexibility and project control.

Project Closure and Review

The final stage of project management involved reviewing the completed Buddy Matcher system against its initial objectives. All core functionalities, including user authentication, property listing, room search, booking, and admin management, were verified.

Final documentation was prepared, and lessons learned during development were documented, including technical challenges, effective solutions, and areas for future improvement. This review ensured successful project completion and provided valuable insights for future web-based application development.

6.2 Major Risks and Contingency Plans

1. Requirement Misinterpretation

Risk: Incorrect understanding of user requirements may result in missing or incorrect features.

Contingency Plan: Regular requirement reviews, clear documentation, and continuous validation during development were performed.

2. Authentication and Authorization Failures

Risk: Improper session handling could allow unauthorized access.

Contingency Plan: Secure Django-based authentication and role-based access control were implemented and tested.

3. Booking Conflicts

Risk: Multiple users attempting to book the same property could cause conflicts.

Contingency Plan: Booking status checks and database constraints were used to prevent duplicate or invalid bookings.

4. Data Privacy and Security

Risk: User or property data could be exposed due to insecure handling.

Contingency Plan: Secure database access, encrypted communication, and strict access controls were enforced.

5. Performance and Scalability Issues

Risk: Increased users could affect system performance.
Contingency Plan: Optimized queries and scalable cloud infrastructure were used to handle increased load.

6. Deployment and Configuration Errors

Risk: Incorrect environment setup could lead to system failure after deployment. **Security/Contingency Plan:** Environment variables were securely managed, and deployment was tested in staging before production release.

6.3 Principal Learning Outcomes

1. **Full-Stack AI Application Development**
   Gained hands-on experience in designing and developing a complete room rental web application.

2. **Secure Authentication and Authorization**
   Learned how to implement role-based authentication and secure session management.

3. **Database Design and Management**
   Developed skills in designing and managing databases for real-world applications.

4. **System Integration and Testing**
   Gained experience in integrating multiple modules and ensuring system reliability through testing.
5. **Performance Optimization and Scalability**
   Understood methods to optimize system performance and design scalable architectures.

6. **Testing and Quality Assurance**
   Learned the importance of structured testing for ensuring system correctness and stability.

7. **Project and Time Management**
   Improved skills in planning, scheduling, and managing project milestones effectively.

8. **User-Centric Design Approach**
   Gained insight into designing intuitive and responsive interfaces focused on user experience.

REFERENCES

[1] Kumar, R., and Sharma, P., "Design and Development of a Web-Based Tourism Management System," International Journal of Computer Science and Information Technology, Vol. 12, No. 3, pp. 45–52, 2020.

[2] Pressman, R. S., & Maxim, B. R. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill.

[3] Singh, A., and Verma, S., "Development of an Online Tourism Management System Using Web Technologies," International Journal of Engineering Research and Technology (IJERT), Vol. 10, No. 5, pp. 210–215, 2021.

[4] Django Software Foundation. Django Web Framework Documentation. Available at: https://docs.djangoproject.com

[5] Bootstrap Documentation. Responsive Web Design Framework. Available at: https://getbootstrap.com

[6] Joshi, N., and Kulkarni, S., "Location-Based Search and Mapping in Web Applications," International Journal of Computer Applications, Vol. 178, No. 7, pp. 25–30, 2019.

[7] Agarwal, M., and Gupta, R., "Online Payment Gateway Integration in Web-Based Systems," Journal of Information Systems and Technology, Vol. 8, No. 4, pp. 60–66, 2021.

[8] Mishra, S., and Tiwari, A., "Role-Based Access Control for Web Applications," International Journal of Advanced Computer Science and Applications (IJACSA), Vol. 11, No. 6, pp. 340–346, 2020.

[9] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2011). Database System Concepts (6th ed.). McGraw-Hill.

[10] Fielding, R. T. (2000). Architectural Styles and the Design of Network-Based Software Architectures. Doctoral Dissertation, University of California, Irvine.

[11] ISO/IEC 25010:2011. Systems and Software Quality Requirements and Evaluation (SQuaRE).

[12] Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann.

[13] World Tourism Organization (UNWTO). Tourism and Digital Transformation Report, 2022.

[14] Python Software Foundation. Python Programming Language Documentation. Available at: https://www.python.org/doc/